

КОНЦЕПЦИЯ КОНЕЧНОГО АВТОМАТА В RTL-ПРОЕКТИРОВАНИИ



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

ПРИ ПАРТНЕРСТВЕ



Занятие №3

26 октября 2025



Автор (разработчик материала) лекции — Силантьев Александр Михайлович

Использование материалов и записи лекции и/или их частей без предварительного согласия не допускается.

По вопросам использования материалов и записи лекции в коммерческих целях необходимо направить обращение в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com.

По вопросам некоммерческого использования материалов и записи лекции обращение может быть направлено в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com, либо на адрес электронной почты автора Силантьева А. М. silantiev@org.miet.ru. Такое обращение обязательно должно содержать описание цели использования.



Александр Силантьев

Руководитель лаборатории НИЛ ЭСК Университета МИЭТ

- Окончил МИЭТ в 2014 году
- 14-летний опыт инженерной деятельности в области проектирования IP-ядер и СнК
- Старший преподаватель института МПСУ МИЭТ
- Организатор первого в России инженерного хакатона по микроэлектронике и системам на кристалле
- С 2014 года организатор семинаров, школ и олимпиад по популяризации электроники среди студентов и школьников



ВИДЫ ПРИСВАИВАНИЙ В SystemVerilog

```
logic a;
logic b;
logic c;
logic d = a & b;
assign c = a & b;
```

НЕПРЕРЫВНОЕ ПРИСВАИВАНИЕ

Непрерывное присваивание может быть использовано только вне программных блоков

```
logic      a_ff;
logic      b_ff;
logic      en;
Logic [1:0] c;

always_ff @(posedge clk) begin
    if (en) begin
        a_ff <= b_ff;
        b_ff <= a_ff;
    end
end

always_comb begin
    c = {a_ff, b_ff};
end
```

ПРОЦЕДУРНОЕ ПРИСВАИВАНИЕ

В отличие от непрерывного присваивания, процедурное присваивание может быть использовано только в программных блоках

ПРОГРАММНЫЕ БЛОКИ

`initial`, `always_ff`, `always_comb`, и т.д.



ПРОЦЕДУРНЫЕ ПРИСВАИВАНИЯ В SystemVerilog

```
logic a;
logic b_ff, c_ff, d_ff;
```

```
always_ff @(posedge clk) begin
```

```
  b_ff <= a;
```

```
  c_ff <= b_ff;
```

```
  d_ff <= c_ff;
```

```
end
```

LHS

RHS

```
logic a;
logic b_ff, c_ff, d_ff;
```

```
always_ff @(posedge clk) begin
```

```
  b_ff = a;
```

```
  c_ff = b_ff;
```

```
  d_ff = c_ff;
```

```
end
```

LHS

RHS

НЕБЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "<="

Производит вычисление **RHS**, запоминает его, и откладывает присваивание вычисленного значения, позволяя выполняться остальным выражениям до завершения присваивания **LHS**.

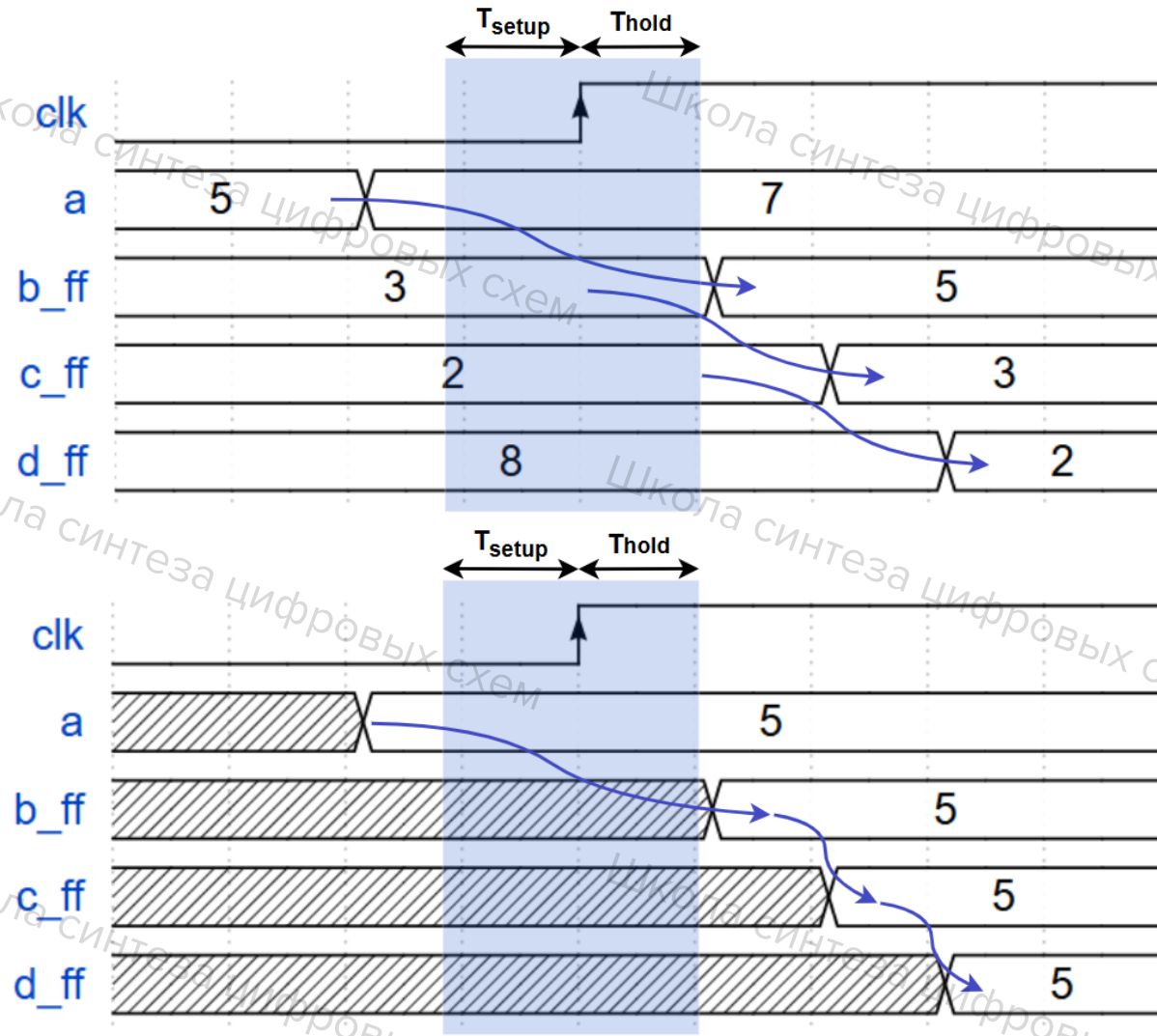
Выражения внутри отдельного программного блока(с точки зрения моделирования) **исполняются последовательно**

БЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "="

Блокирует исполнение дальнейших выражений до завершения вычисления **RHS** и присвоения вычисленного результата **LHS** (иными словами, это присваивание из мира программирования).



ПРОЦЕДУРНЫЕ ПРИСВАИВАНИЯ В SystemVerilog



НЕБЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "<="

1. Вычисляется значение **RHS** первого присваивания (3->5). Присваивание **b_ff** откладывается на потом.
2. Поскольку **b_ff** еще не присвоили значение 5, результатом **RHS** для **c_ff** становится текущее значение **b_ff** (2->3). Присваивание **c_ff** откладывается на потом.
3. Аналогичным образом вычисляется **RHS** третьего присваивания (8->2). Присваивание этого значения также откладывается на потом.

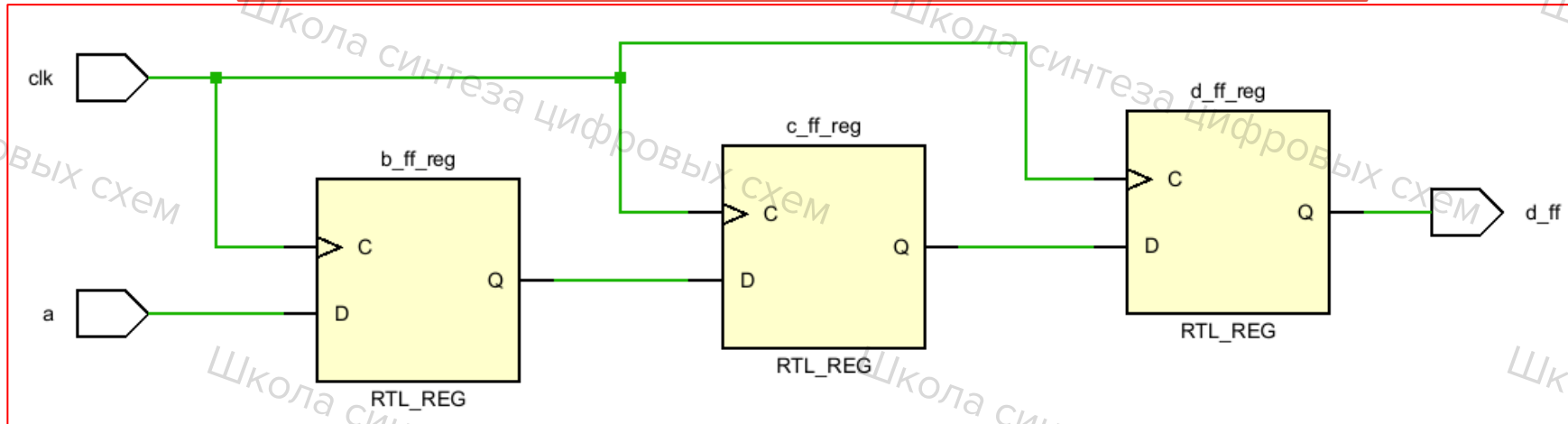
БЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "="

1. Вычисляется **RHS** первого присваивания программного блока **a** (5). Затем, вычисленное значение записывается в **LHS** первого присваивания - **b_ff** (5).
2. Далее вычисляется **RHS** следующего присваивания - **b_ff**, которое к этому моменту уже равно 5.
3. Поскольку вычисленное **RHS** равняется 5 то **LHS** второго присваивания - **c_ff** тоже становится равным 5.
4. Аналогичным образом **d_ff** тоже становится равным 5.

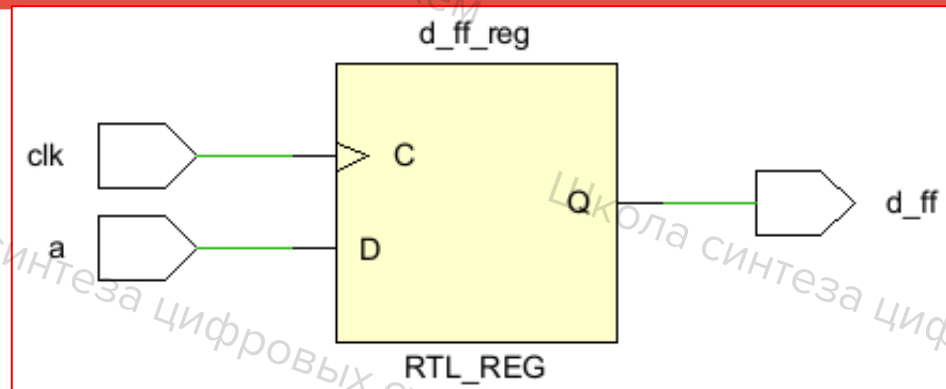


ПРОЦЕДУРНЫЕ ПРИСВАИВАНИЯ В SystemVerilog

НЕБЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "<="



БЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "="





ПРОЦЕДУРНЫЕ ПРИСВАИВАНИЯ В SystemVerilog

НЕБЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "<="

```
logic [2:0] x_ff, y_ff;  
  
always_ff @(posedge clk) begin  
  
    x_ff <= x_ff + 1'b1;  
  
    y_ff <= y_ff + 1'b1;  
end
```

~

БЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "="

```
logic [2:0] x_ff, y_ff;  
  
always_ff @(posedge clk) begin  
  
    x_ff = x_ff + 1'b1;  
  
    y_ff = y_ff + 1'b1;  
end
```



ПРОЦЕДУРНЫЕ ПРИСВАИВАНИЯ В SystemVerilog

НЕБЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "<="

```
logic [2:0] x_ff, y_ff;

always_ff @(posedge clk) begin

    x_ff <= x_ff + 1'b1;

    y_ff <= y_ff + 1'b1;
end
```

~

БЛОКИРУЮЩЕЕ ПРИСВАИВАНИЕ "="

```
logic [2:0] x_ff, y_ff;

always_ff @(posedge clk) begin

    x_ff = x_ff + 1'b1;

    y_ff = y_ff + 1'b1;
end
```

```
logic [2:0] x_ff, y_ff;

always_ff @(posedge clk) begin

    x_ff <= x_ff + 1'b1;

    y_ff <= x_ff + 1'b1;
end
```

~

```
logic [2:0] x_ff, y_ff;

always_ff @(posedge clk) begin

    x_ff = x_ff + 1'b1;

    y_ff = x_ff;
end
```



parameter и localparam

```
module fifo #(
    parameter DEPTH = 1024,
    parameter DATA_WIDTH = 8
) (
    input logic clk,
    input logic [DATA_WIDTH-1:0] data_in,
    output logic [DATA_WIDTH-1:0] data_out
);
    localparam ADDR_WIDTH = $clog2(DEPTH);

    // Использование localparam для объявления сигнала
    logic [ADDR_WIDTH-1:0] write_ptr;
    logic [ADDR_WIDTH-1:0] read_ptr;

    // Использование parameter для определения размера памяти
    logic [DATA_WIDTH-1:0] memory [0:DEPTH-1];

    // ... остальная логика ...
endmodule
```

parameter — это константа, которую можно переопределить извне. Используется для создания настраиваемых модулей



parameter и localparam

```
module fifo #(
    parameter DEPTH = 1024,
    parameter DATA_WIDTH = 8
) (
    input logic clk,
    input logic [DATA_WIDTH-1:0] data_in,
    output logic [DATA_WIDTH-1:0] data_out
);
    localparam ADDR_WIDTH = $clog2(DEPTH);

    // Использование localparam для объявления сигнала
    logic [ADDR_WIDTH-1:0] write_ptr;
    logic [ADDR_WIDTH-1:0] read_ptr;

    // Использование parameter для определения размера памяти
    logic [DATA_WIDTH-1:0] memory [0:DEPTH-1];

    // ... остальная логика ...
endmodule
```

parameter — это константа, которую можно переопределить извне. Используется для создания настраиваемых модулей

localparam — это локальная константа внутри модуля. Её значение не может быть изменено извне



instance модуля с parameter

```
module fifo_top #(
    parameter DEPTH = 1024,
    parameter DATA_WIDTH = 8
) (
    input logic clk,
    input logic [DATA_WIDTH-1:0] data_i,
    output logic [DATA_WIDTH-1:0] data_o
);
```

```
// ... остальная логика ...
```

```
module fifo_0 #(
    .DEPTH      ( DEPTH      ),
    .DATA_WIDTH ( DATA_WIDTH )
) (
    .clk        ( clk        ),
    .data_in    ( data_i     ),
    .data_out   ( data_o     )
);
```

```
// ... остальная логика ...
```

```
endmodule
```

Подключение параметризуемого экземпляра fifo_0
модуля fifo



`define и `ifdef

``define` — это директива препроцессора, которая создаёт текстовый макрос. Препроцессор подставляет текст вместо имени макроса

```
`define DATA_BUS_WIDTH 32
`define DEBUG

module spu (
    input logic [`DATA_BUS_WIDTH-1:0] data_in,
    output logic result
);

    logic [`DATA_BUS_WIDTH-1:0] internal_data;

    `ifdef DEBUG
        $display("Debug mode: Starting simulation");
        // ... DEBUG код ...
    `endif

    // ... остальная логика ...
endmodule
```



`define и `ifdef

```
`define DATA_BUS_WIDTH 32
`define DEBUG

module spu (
    input logic [`DATA_BUS_WIDTH-1:0] data_in,
    output logic result
);

    logic [`DATA_BUS_WIDTH-1:0] internal_data;

    `ifdef DEBUG
        $display("Debug mode: Starting simulation");
        // ... DEBUG код ...
    `endif

    // ... остальная логика ...
endmodule
```

``define` — это директива препроцессора, которая создаёт текстовый макрос. Препроцессор подставляет текст вместо имени макроса

``ifdef` — это директива препроцессора, которая позволяет пропускать код в зависимости от того, определен ли макрос



typedef

```
// Создание псевдонима для существующего типа
typedef logic [31:0] bus32_t;
// Создание псевдонима для массива
typedef logic [7:0] byte_array_t [0:31];

// Объявления с пользовательскими типами
bus32_t program_counter;
byte_array_t rf_memory;
```

typedef — это механизм, который позволяет дать новое имя типу данных. Он сделан для удобства и читаемости кода



enum

```
// RED = 0, YELLOW = 1, GREEN = 2
enum {RED, YELLOW, GREEN} light;

// Определение состояний конечного автомата (FSM)
typedef enum logic [1:0] {
    IDLE_S = 2'b00, // Явное присваивание значений
    READ_S = 2'b01,
    WRITE_S = 2'b10,
    DONE_S = 2'b11
} state, nextstate;
```

enum — способ задать набор именованных состояний или значений вместо использования сырых чисел. При такой записи значения задаются последовательно, начиная с нуля



enum

```
// RED = 0, YELLOW = 1, GREEN = 2
enum {RED, YELLOW, GREEN} light;

// Определение состояний конечного автомата (FSM)
typedef enum logic [1:0] {
    IDLE_S = 2'b00, // Явное присваивание значений
    READ_S = 2'b01,
    WRITE_S = 2'b10,
    DONE_S = 2'b11
} state, nextstate;
```

enum — способ задать набор именованных состояний или значений вместо использования сырых чисел. При такой записи значения задаются последовательно, начиная с нуля

Часто enum пишется через typedef. Например, так описывают состояния конечных автоматов. При такой записи значения задаются явно



struct

struct — способ логически объединить несколько сигналов в один объект с именованными полями

```
// Определение структуры
typedef struct {
    logic [7:0] addr;
    logic [31:0] data;
    logic      valid;
    logic      rw;
} transaction;

transaction.addr  = 8'h0;
transaction.data  = 32'hFADEBABA;
transaction.valid = 1'h1;
transaction.rw    = 1'h1;
```




struct

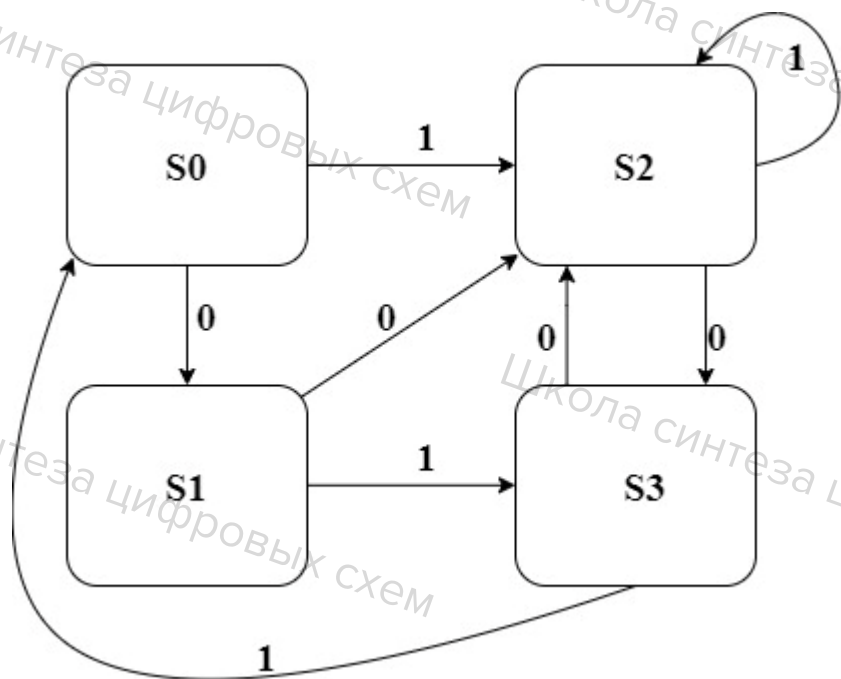
struct — способ логически объединить несколько сигналов в один объект с именованными полями

```
// Определение структуры
typedef struct {
    logic [7:0] addr;
    logic [31:0] data;
    logic      valid;
    logic      rw;
} transaction;

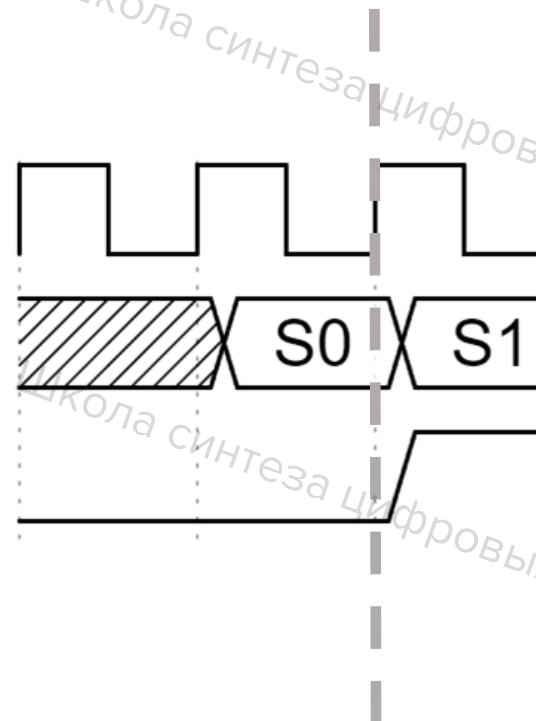
transaction.addr  = 8'h0;
transaction.data  = 32'hFADEBABA;
transaction.valid = 1'h1;
transaction.rw    = 1'h1;
```

Обращение к полям структуры

КОНЕЧНЫЕ АВТОМАТЫ

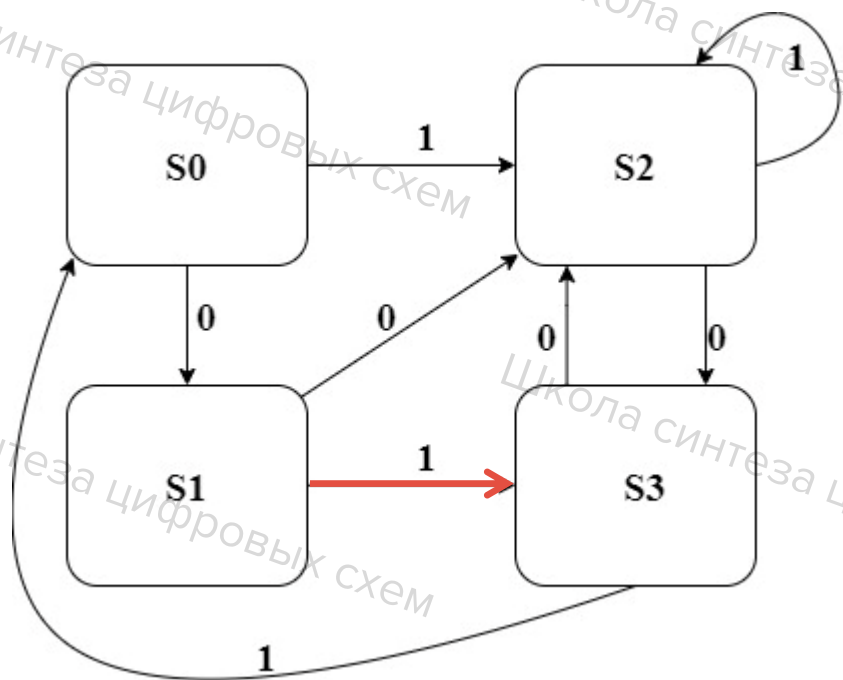


CLK
STATE
IN

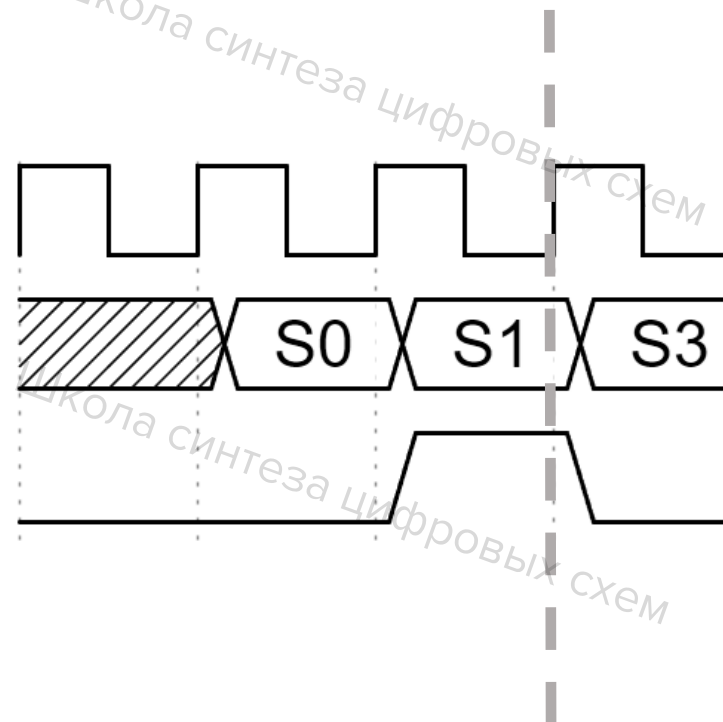




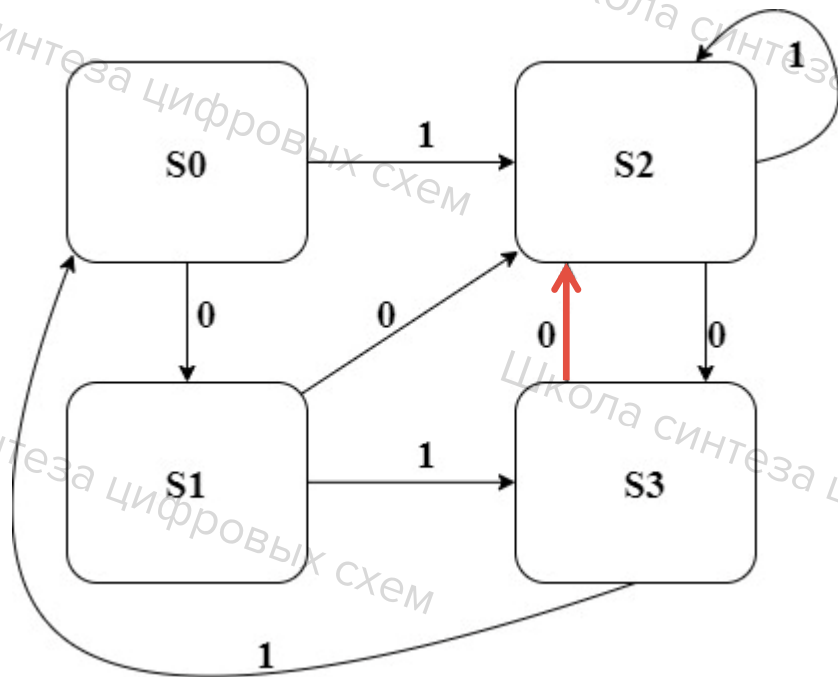
КОНЕЧНЫЕ АВТОМАТЫ



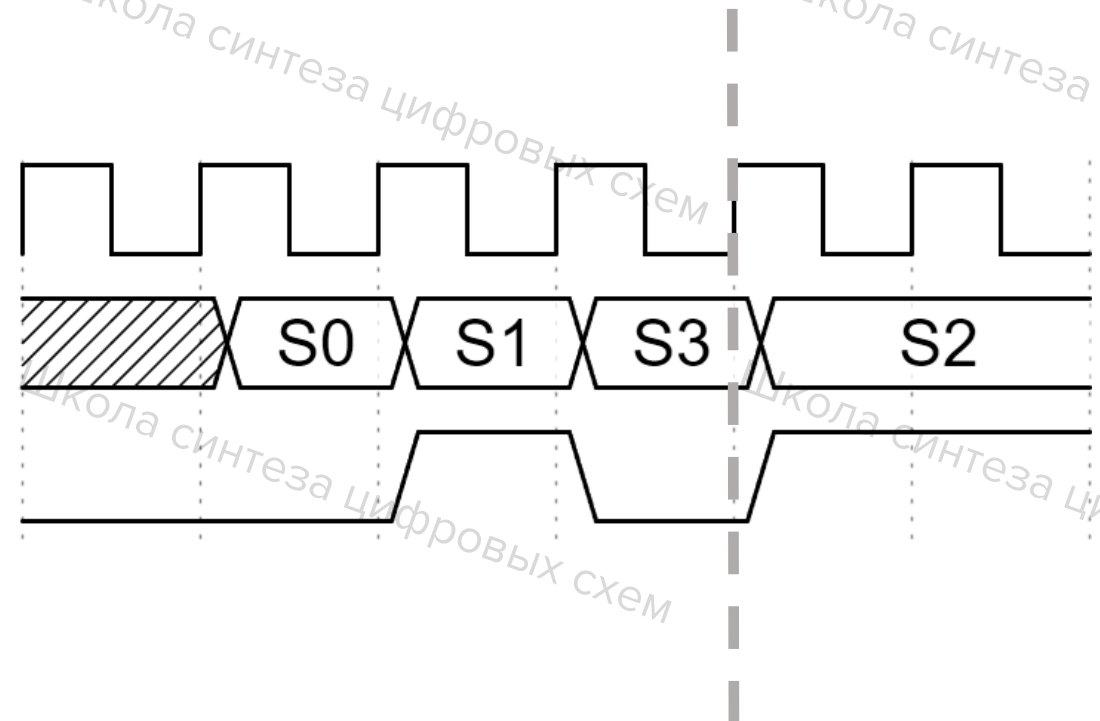
CLK
STATE
IN



КОНЕЧНЫЕ АВТОМАТЫ

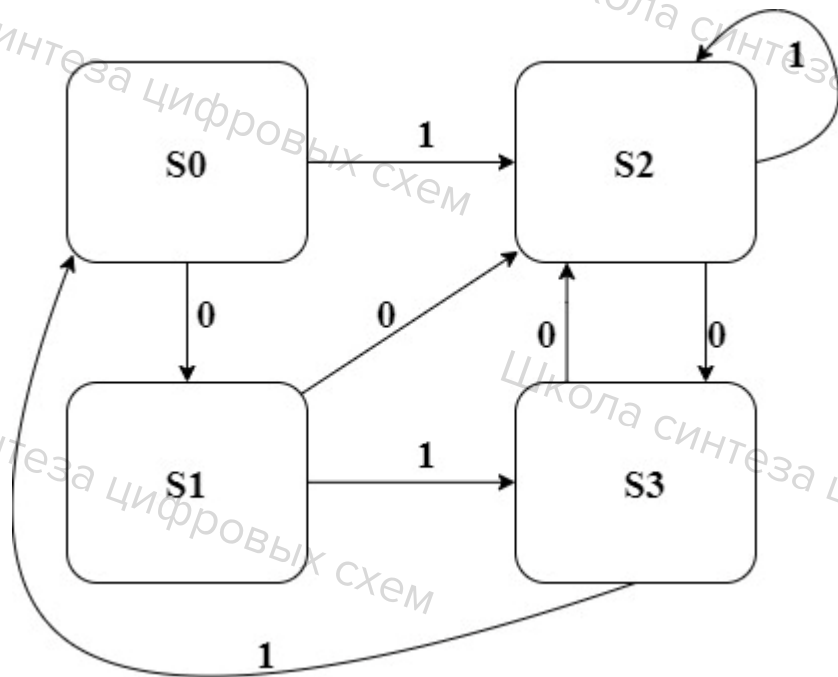


CLK
STATE
IN





КОНЕЧНЫЕ АВТОМАТЫ. HDL-ОПИСАНИЕ

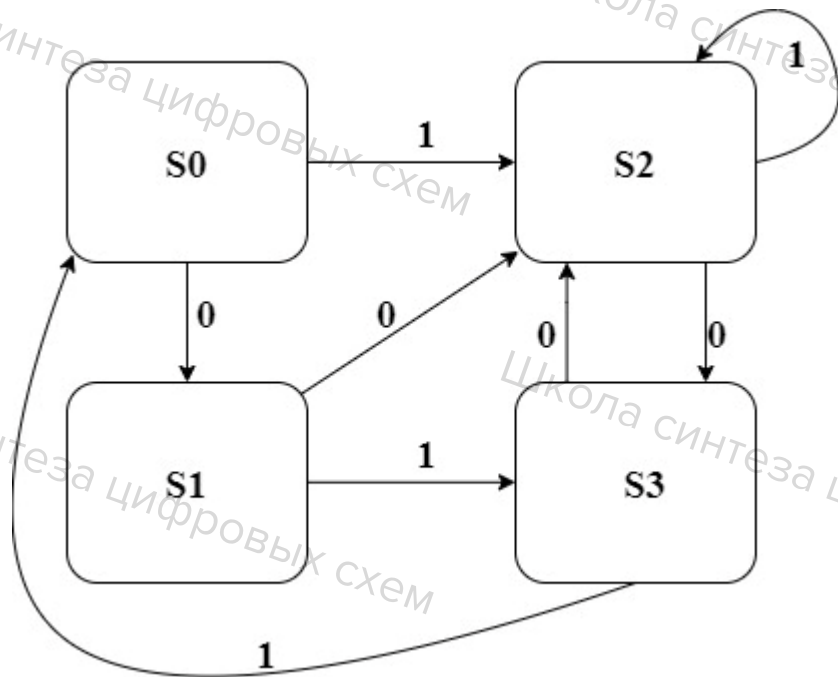


```

logic [1:0] state;
always_ff @(posedge CLK) begin
    if (rst)
        state <= 2'd0;
    else begin
        case (state)
            2'd0: if (in == 1'b1) state <= 2'd2;
                  else state <= 2'd1;
            2'd1: if (in == 1'b1) state <= 2'd3;
                  else state <= 2'd2;
            ...
        endcase
    end
end
    
```



КОНЕЧНЫЕ АВТОМАТЫ. HDL-ОПИСАНИЕ



```

logic [1:0] state;
always_ff @(posedge CLK) begin
    if (rst)
        state <= 2'd0;
    else begin
        case (state)
            2'd0: if (in == 1'b1) state <= 2'd2;
                  else state <= 2'd1;
            2'd1: if (in == 1'b1) state <= 2'd3;
                  else state <= 2'd2;
            ...
        endcase
    end
end
    
```

Рабочий, НО очень плохой стиль описания конечных автоматов



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```
wire enable;  
wire fsm_in, moore_fsm_out, mealy_fsm_out;  
  
// Generate a strobe signal 3 times a second  
strobe_gen #(  
    .clk_mhz (clk_mhz),  
    .strobe_hz (3)  
) i_strobe_gen (  
    .strobe (enable),  
    .*  
);
```

← Параметр, отвечающий за частоту следования стробов в герцах



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```

wire enable;
wire fsm_in, moore_fsm_out, mealy_fsm_out;

// Generate a strobe signal 3 times a second
strobe_gen #(
    .clk_mhz (clk_mhz),
    .strobe_hz (3)
) i_strobe_gen (
    .strobe (enable),
    .*
);
    
```

Параметр, отвечающий за частоту следования стробов в герцах

Генерация строба частотой в 3 Гц. Будет использоваться в управлении сдвиговым регистром



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

Сдвиговый регистр, совпадающий по разрядности
с количеством светодиодов на вашей плате

```
shift_reg # (.depth (w_led)
) i_shift_reg (
    .en      ( enable ),
    .seq_in  ( |key    ),
    .seq_out  ( fsm_in  ),
    .par_out  ( led     ),
    .*
);
```



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```
shift_reg # (.depth (w_led)
) i_shift_reg (
  .en      ( enable ),
  .seq_in  ( |key   ),
  .seq_out ( fsm_in ),
  .par_out ( led    ),
  .*
);
```

Сдвиговый регистр, совпадающий по разрядности с количеством светодиодов на вашей плате

Сдвиг происходит по стробу **enable** три раза в секунду



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```
shift_reg # (.depth (w_led)
) i_shift_reg (
    .en      ( enable ),
    .seq_in  ( |key   ),
    .seq_out ( fsm_in ),
    .par_out ( led    ),
    .*
);
```

Сдвиговый регистр, совпадающий по разрядности с количеством светодиодов на вашей плате

Сдвиг происходит по стробу **enable** три раза в секунду

На вход сдвигового регистра подается результат операции логическое ИЛИ от кнопок на плате. Если в момент, когда enable равен 1, хотя бы одна кнопка будет нажата, то на вход сдвигового регистра будет подана 1



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```
shift_reg # (.depth (w_led)
) i_shift_reg (
    .en      ( enable ),
    .seq_in  ( |key   ),
    .seq_out ( fsm_in ),
    .par_out ( led    ),
    .*
);
```

Сдвиговый регистр, совпадающий по разрядности с количеством светодиодов на вашей плате

Сдвиг происходит по стробу **enable** три раза в секунду

На вход сдвигового регистра подается результат операции логическое ИЛИ от кнопок на плате. Если в момент, когда enable равен 1, хотя бы одна кнопка будет нажата, то на вход сдвигового регистра будет подана 1

Выход сдвигового регистра. Подается на конечный автомат как входное воздействие



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```
shift_reg # (.depth (w_led)
) i_shift_reg (
  .en      ( enable ),
  .seq_in  ( |key   ),
  .seq_out ( fsm_in ),
  .par_out ( led    ),
  .*
);
```

Сдвиговый регистр, совпадающий по разрядности с количеством светодиодов на вашей плате

Сдвиг происходит по стробу **enable** три раза в секунду

На вход сдвигового регистра подается результат операции логическое ИЛИ от кнопок на плате. Если в момент, когда enable равен 1, хотя бы одна кнопка будет нажата, то на вход сдвигового регистра будет подана 1

Выход сдвигового регистра. Подается на конечный автомат как входное воздействие

Шина, отражающая состояние всех триггеров сдвигового регистра для визуализации на светодиодах



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```
snail_moore_fsm i_moore_fsm
(.en(enable), .a(fsm_in), .y(moore_fsm_out), .*);
snail_mealy_fsm i_mealy_fsm
(.en(enable), .a(fsm_in), .y(mealy_fsm_out), .*);

always_comb
begin
    case ({ mealy_fsm_out, moore_fsm_out })
        2'b00: abcdefgh = 8'b0000_0000;
        2'b01: abcdefgh = 8'b1100_0110; // Moore only
        2'b10: abcdefgh = 8'b0011_1010; // Mealy only
        2'b11: abcdefgh = 8'b1111_1110;
    endcase
    digit = w_digit' (1);
end
```

Два идентичных по функционалу конечных автомата, написанных в разных форматах. На вход приходит состояние кнопок с сдвигового регистра. На выходе была распознана последовательность нажатий или нет



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

```

snail_moore_fsm i_moore_fsm
(.en(enable), .a(fsm_in), .y(moore_fsm_out), .*);
snail_mealy_fsm i_mealy_fsm
(.en(enable), .a(fsm_in), .y(mealy_fsm_out), .*);

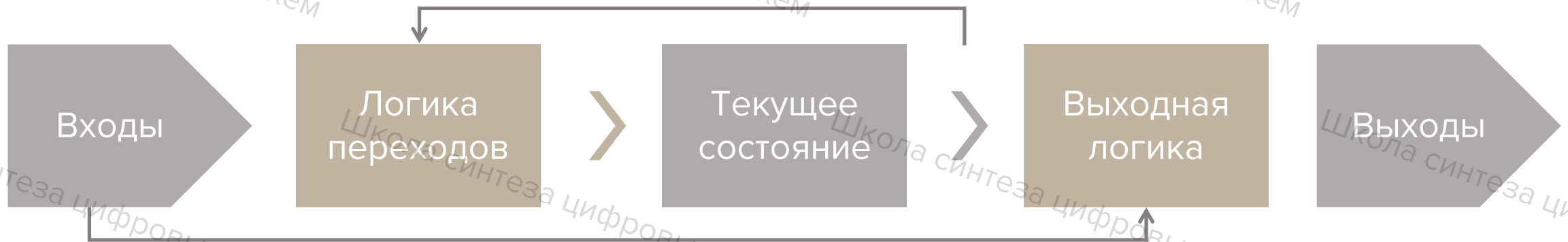
always_comb
begin
    case ({ mealy_fsm_out, moore_fsm_out })
        2'b00: abcdefgh = 8'b0000_0000;
        2'b01: abcdefgh = 8'b1100_0110; // Moore only
        2'b10: abcdefgh = 8'b0011_1010; // Mealy only
        2'b11: abcdefgh = 8'b1111_1110;
    endcase
    digit = w_digit' (1);
end
    
```

Два идентичных по функционалу конечных автомата, написанных в разных форматах. На вход приходит состояние кнопок с сдвигового регистра. На выходе была распознана последовательность нажатий или нет

На один семисегментный индикатор выводится информация о состоянии конечных автоматов



КОНЕЧНЫЕ АВТОМАТЫ. АВТОМАТ МИЛИ





УПРАЖНЕНИЕ 1_10_SNAIL_FSM. SNAIL_MEALY_FSM

```
typedef enum bit
{
    S0 = 1'd0,
    S1 = 1'd1
}
state_e;
state_e state, next_state;
```

Описание кодирования состояний
конечного автомата

```
// State register
always_ff @ (posedge clk or posedge rst)
    if (rst)
        state <= S0;
    else if (en)
        state <= next_state;
```



УПРАЖНЕНИЕ 1_10_SNAIL_FSM. SNAIL_MEALY_FSM

```
typedef enum bit
{
    S0 = 1'd0,
    S1 = 1'd1
}
state_e;
state_e state, next_state;
```

Описание кодирования состояний
конечного автомата

```
// State register
always_ff @ (posedge clk or posedge rst)
    if (rst)
        state <= S0;
    else if (en)
        state <= next_state;
```

Описание триггера, хранящего в себе состояние
конечного автомата. Оно обновляется каждый
такт из next_state.



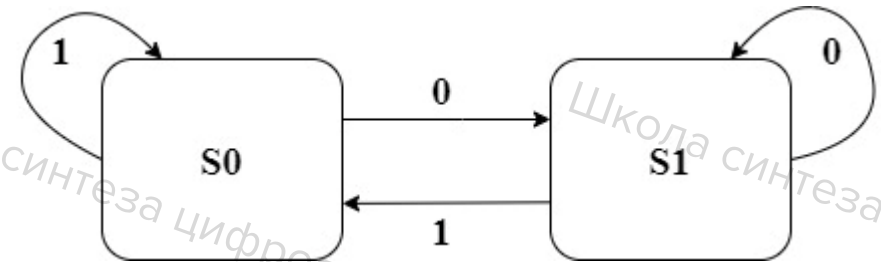
УПРАЖНЕНИЕ 1_10_SNAIL_FSM. SNAIL_MEALY_FSM

```
always_comb
begin
    next_state = state;

    case (state)
        S0: if (~a) next_state = S1;
        S1: if ( a) next_state = S0;
    endcase
end

// Output logic based on current state and inputs
assign y = (a & state == S1);
```

Описание логики переходов между состояниями.
В данном случае из S0 в S1 произойдёт переход если вход равен 0. Из S1 в S0 если вход равен 1





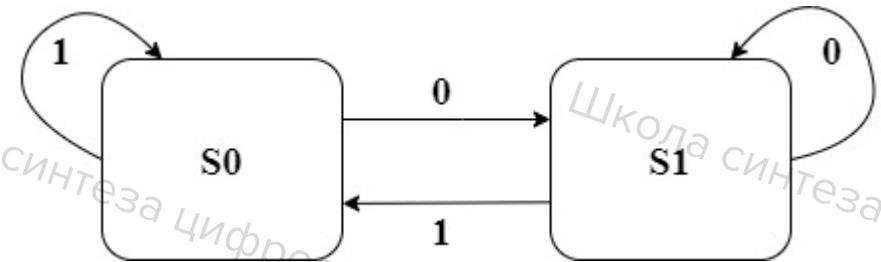
УПРАЖНЕНИЕ 1_10_SNAIL_FSM. SNAIL_MEALY_FSM

```
always_comb
begin
    next_state = state;

    case (state)
    S0: if (~a) next_state = S1;
    S1: if ( a) next_state = S0;
    endcase
end

// Output logic based on current state and inputs
assign y = (a & state == S1);
```

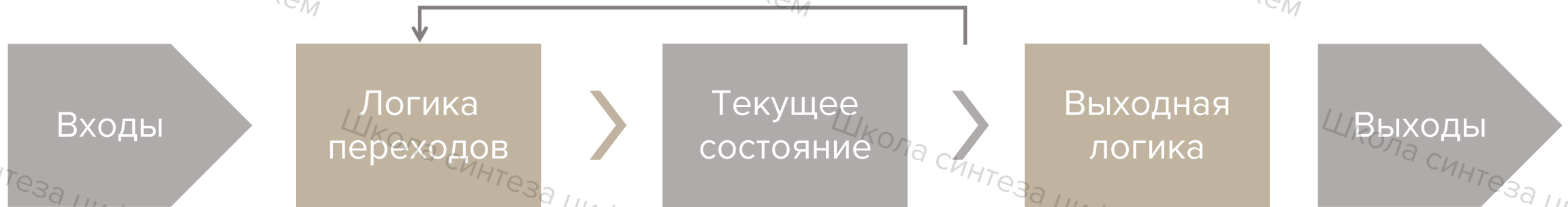
Описание логики переходов между состояниями. В данном случае из S0 в S1 произойдёт переход если вход равен 0. Из S1 в S0 если вход равен 1



Выход конечного автомата будет равен 1 только в момент когда состояние S1 и вход равен 1. По-другому это переход из S1 в S0



КОНЕЧНЫЕ АВТОМАТЫ. АВТОМАТ МУРА





УПРАЖНЕНИЕ 1_10_SNAIL_FSM. SNAIL_MOORE_FSM

```
always_comb
begin
    next_state = state;
    case (state)
        S0: if (~a) next_state = S1;
        S1: if ( a) next_state = S2;

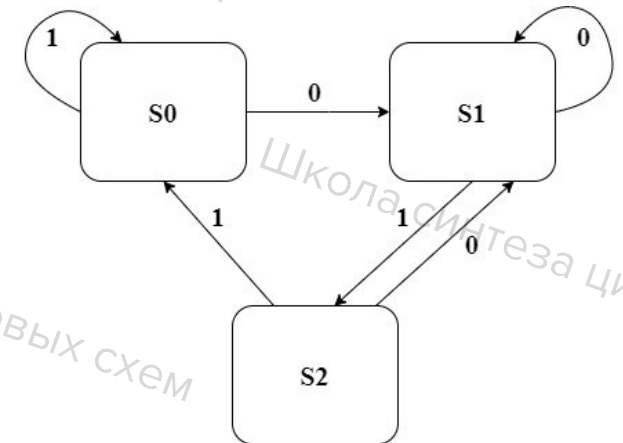
        // S2: next_state = a ? S0 : S1;

        S2: if ( a)
            next_state = S0;
            else
            next_state = S1;
    endcase
end
```

```
// Output logic based on current state
assign y = (state == S2);
```

Добавление отдельного состояния.
Интересно, что если в состоянии S2 придет 0, то мы перейдем в состояние S1 проскочив S0

```
typedef enum bit [1:0]
{
    S0 = 2'd0,
    S1 = 2'd1,
    S2 = 2'd2
}
state_e;
```





УПРАЖНЕНИЕ 1_10_SNAIL_FSM. SNAIL_MOORE_FSM

```
always_comb
begin
    next_state = state;
    case (state)
        S0: if (~a) next_state = S1;
        S1: if ( a) next_state = S2;

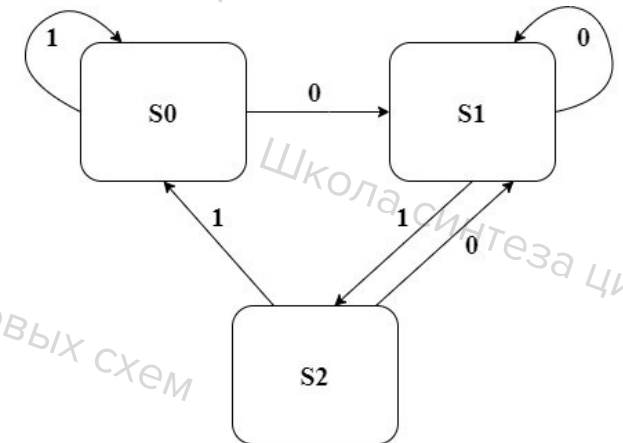
        // S2: next_state = a ? S0 : S1;

        S2: if ( a)
            next_state = S0;
            else
            next_state = S1;
    endcase
end
```

```
// Output logic based on current state
assign y = (state == S2);
```

Добавление отдельного состояния.
Интересно, что если в состоянии S2 придет 0, то мы перейдем в состояние S1 проскочив S0

```
typedef enum bit [1:0]
{
    S0 = 2'd0,
    S1 = 2'd1,
    S2 = 2'd2
}
state_e;
```



Выход конечного автомата привязан только к состоянию конечного автомата



УПРАЖНЕНИЕ 1_10_SNAIL_FSM

- Модифицируйте конечный автомат **Мура** так, чтобы он распознавал последовательность входного сигнала «1, 0, 1, 1, 0».
- Модифицируйте конечный автомат **Мили** так, чтобы он распознавал последовательность входного сигнала «1, 0, 1, 1, 0».
- В первую очередь нарисуйте на листочке граф конечного автомата.
С начала идея => потом код.
- Обратите внимание, что количество состояний станет больше, а значит увеличится **разрядность** значений, кодирующих состояние.
- Обратите внимание, что неправильная последовательность нажатий может вас привести **не в начальное состояние, а в промежуточное.**

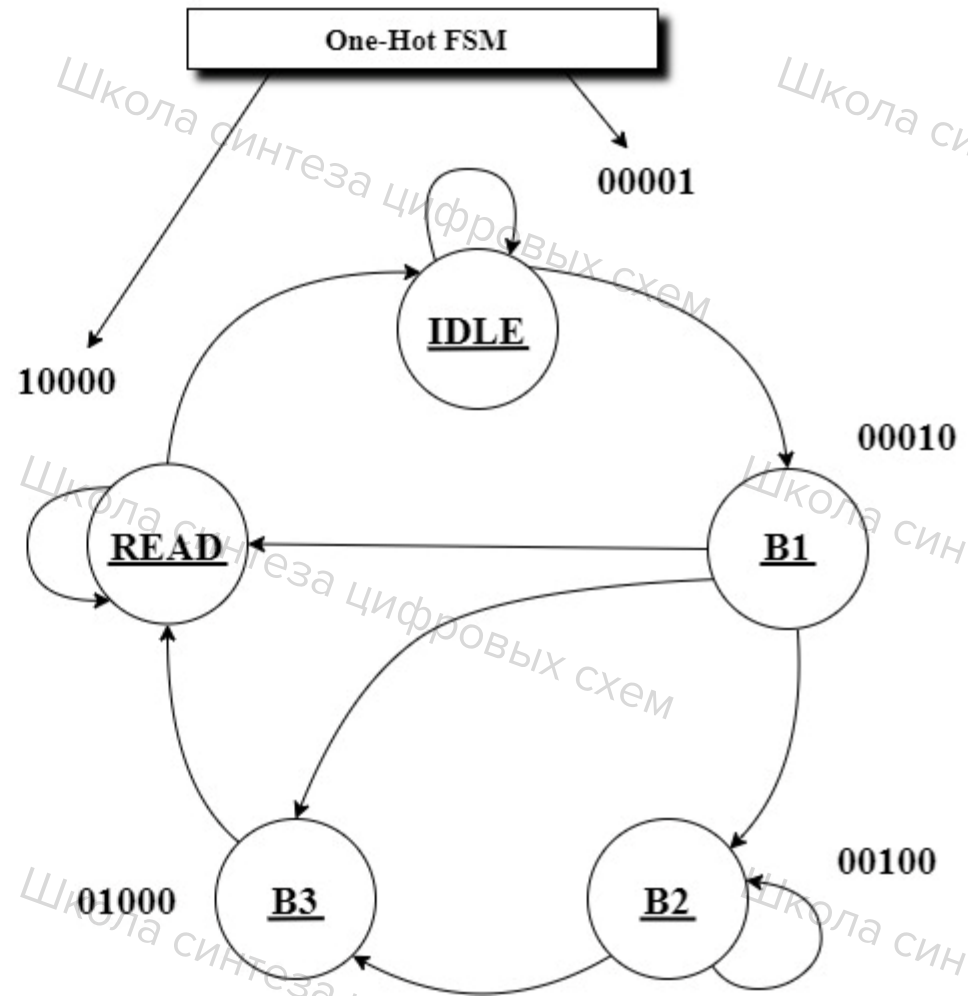
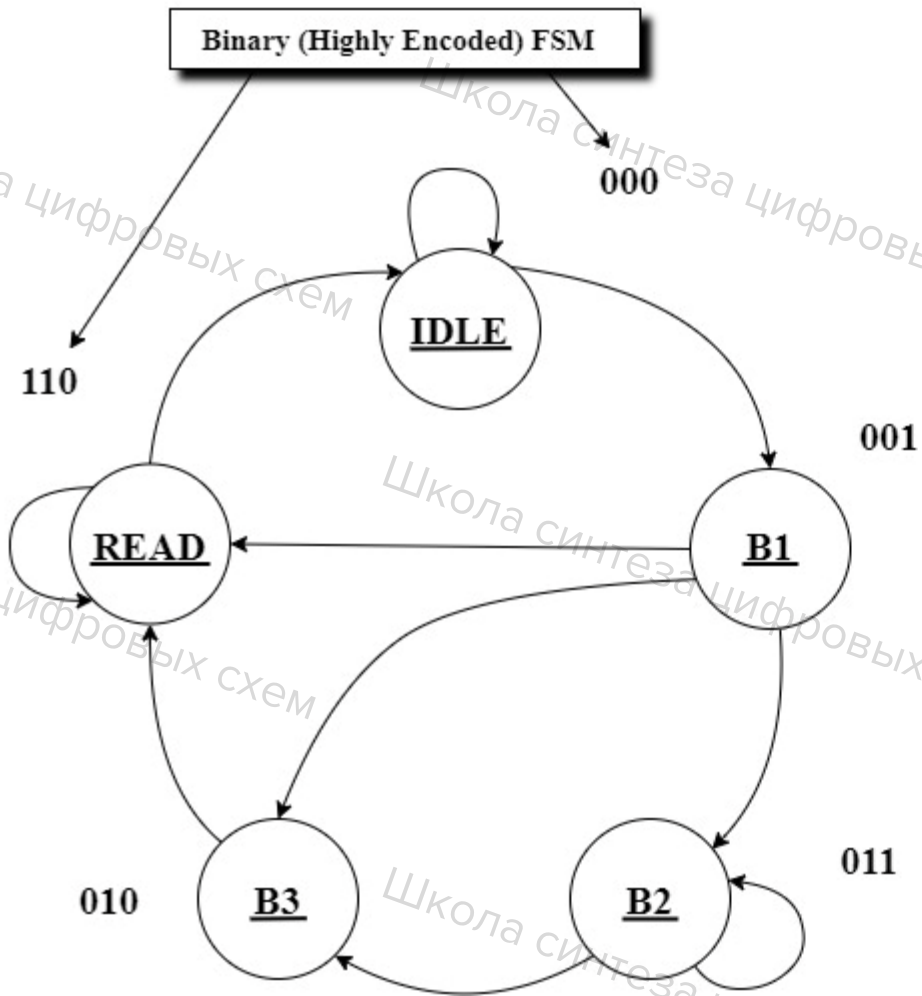


КОНЕЧНЫЕ АВТОМАТЫ. ИСТОЧНИКИ ИНФОРМАЦИИ

Материалы для этой части презентации:

- Clifford E. Cummings The Fundamentals of Efficient Synthesizable Finite State Machine Design using NC-Verilog and BuildGates
- Clifford E. Cummings Finite State Machine (FSM) Design & Synthesis using SystemVerilog — Part I

КОНЕЧНЫЕ АВТОМАТЫ. КОДИРОВАНИЕ





КОНЕЧНЫЕ АВТОМАТЫ. ИСТОЧНИКИ ИНФОРМАЦИИ

- Описание конечного автомата должно быть легко модифицируемым.
- Описание конечного автомата должно быть компактным.
- Описание конечного автомата должно быть легким для понимания.
- Описание конечного автомата должно облегчать отладку.
- Описание конечного автомата должно давать эффективные результаты синтеза.

Больше кода = больше ошибок



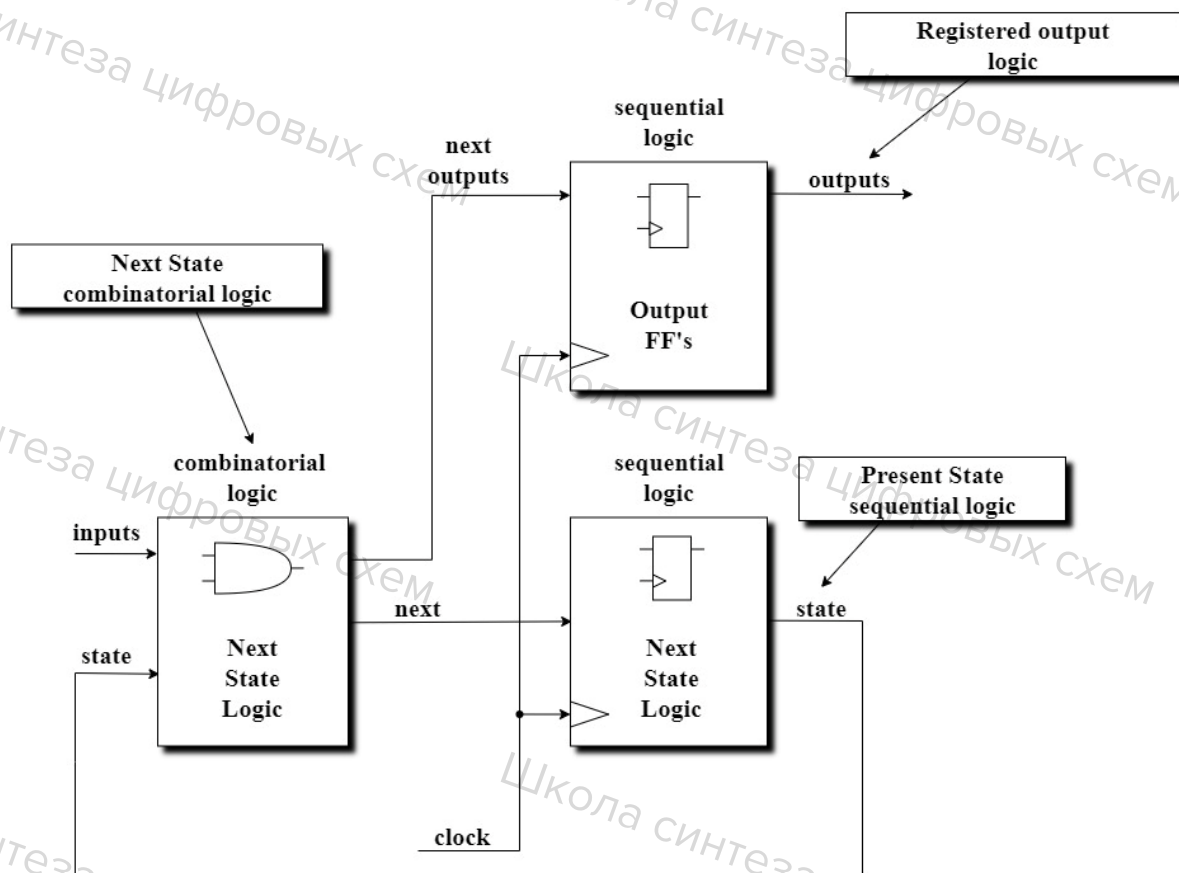
КОНЕЧНЫЕ АВТОМАТЫ. ИСТОЧНИКИ ИНФОРМАЦИИ

- Описание конечного автомата должно быть легко модифицируемым.
- Описание конечного автомата должно быть компактным.
- Описание конечного автомата должно быть легким для понимания.
- Описание конечного автомата должно облегчать отладку.
- Описание конечного автомата должно давать эффективные результаты синтеза.

А больше непонятого кода = УЖАС



КОНЕЧНЫЕ АВТОМАТЫ. ONE ALWAYS BLOCK FSM



```
module fsm
  (... ports ...);
  import fsm_pkg::*;
  state_e state;
```

```
always_ff @(posedge clk, negedge rst_n)
  if (!rst_n) begin
    state <= IDLE;
    <outputs> <= '0;
  end
  else begin
    state <= XXX;
    <outputs> <= '0;
    case (state)
      IDLE : if (go) begin
        rd <= '1;
      end
      else
        READ : begin
          rd <= '1;
        end
      ...
    endcase
  end
endmodule
```

Present state
being registered

Next outputs
being registered

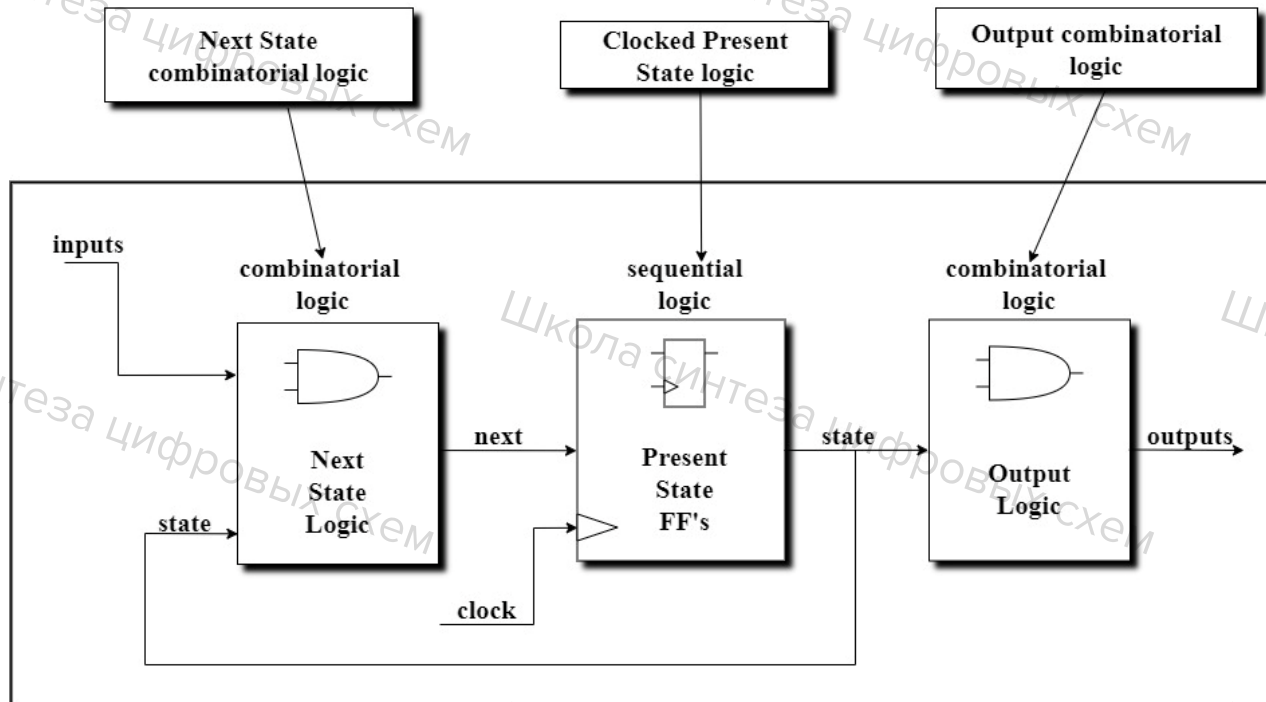
state <= READ;

state <= IDLE;

Next states being
registered



КОНЕЧНЫЕ АВТОМАТЫ. TWO ALWAYS BLOCK FSM



```
module fsm
    (... ports ...);
    import fsm_pkg::*;
    state_e state, next;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= IDLE;
        else state <= next;

    always_comb begin
        next = XXX;
        rd = '0;
        ds = '0;

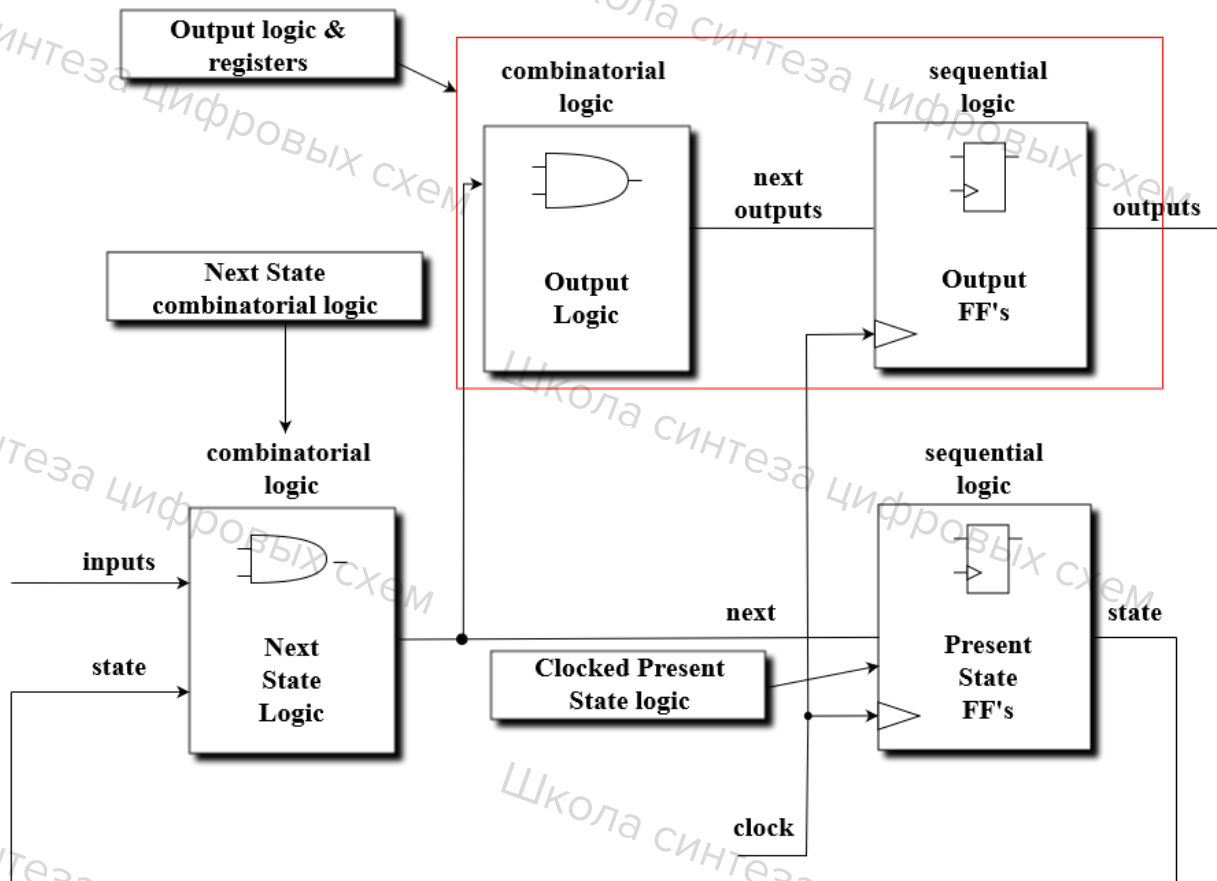
        case (state)
            IDLE : if (go) next = READ;
                  else next = IDLE;
            READ : begin
                      rd = '1;
                      next = DLY;
                  end
            ...
        endcase
    end
endmodule
```

Clocked present
state logic

Next & output
combinational
logic



КОНЕЧНЫЕ АВТОМАТЫ. THREE ALWAYS BLOCK FSM



```
module fsm
  (... ports ...);
  import fsm_pkg::*;
  state_e state, next;
```

Clocked present
state logic

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) state <= IDLE;
  else state <= next;
```

always_comb
next logic

```
always_comb begin
  next = XXX;
  case (state)
    IDLE: if (go) next = READ;
          else next = IDLE;
    READ: next = DLY;
    ...
  endcase
end
```

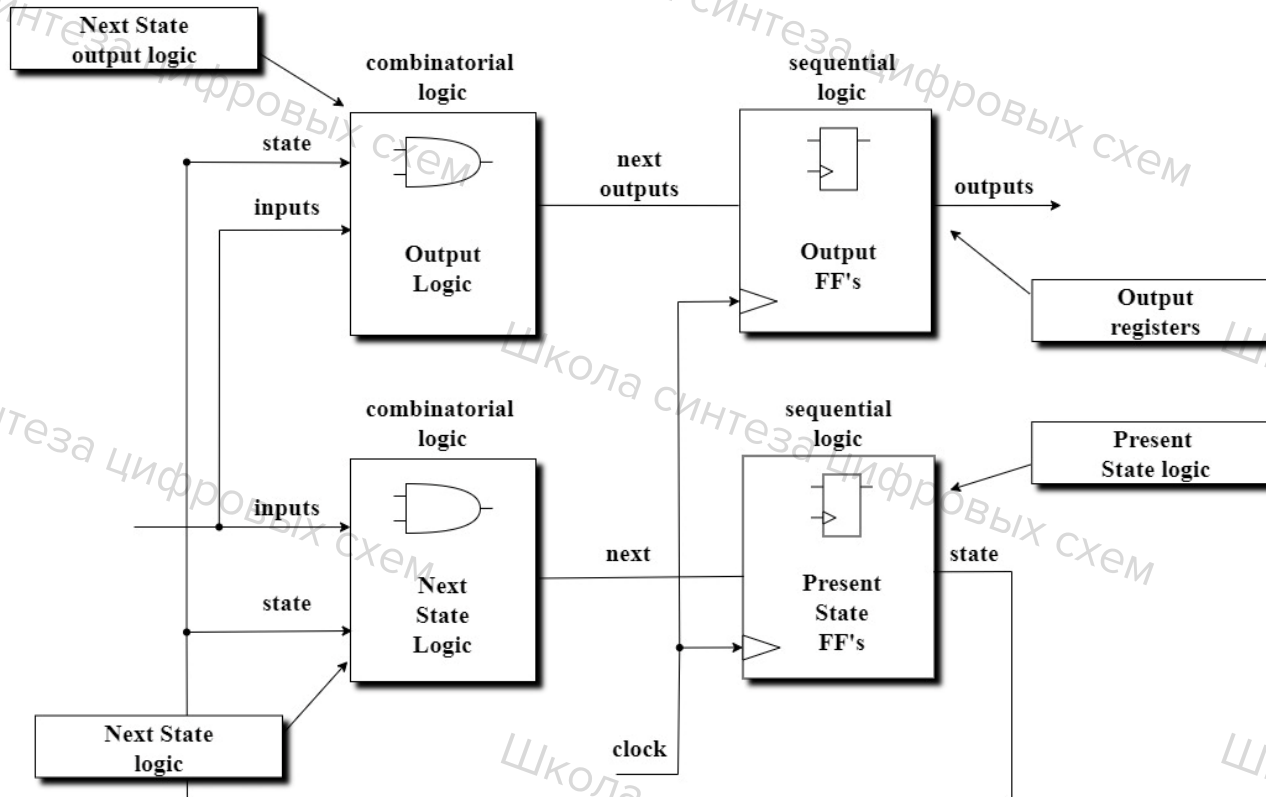
always_ff
next-output logic
& registers

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    rd <= '0;
    ds <= '0;
  end else begin
    rd <= '0;
    ds <= '0;
    case (state)
      READ: rd <= '1; // READ
      ...
    endcase
  end
end
endmodule
```

When next is
READ, output will
be rd <= 1



КОНЕЧНЫЕ АВТОМАТЫ. FOUR ALWAYS BLOCK FSM



```
module fsm
  (... ports ...);
  import fsm_pkg::*;
  state_e state, next;
```

Clocked present
state logic

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) state <= IDLE;
  else state <= next;
```

always_comb
next logic

```
always_comb begin
  next = XXX;
  case (state)
    IDLE: if (go) next = READ;
          else next = IDLE;
    ...
  endcase
end
```

always_comb
next-output logic

```
always_comb begin
  n_rd = '0;
  n_ds = '0;
  case (state)
    READ: n_rd = '1; // READ
    ...
  endcase
end
```

next-outputs

always_ff
output registers

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin ...
  end else begin
    rd <= n_rd;
    ds <= n_ds;
  end
endmodule
```




КОНЕЧНЫЕ АВТОМАТЫ. ENUM

```
package fsm1_pkg;  
    typedef enum {IDLE, READ, DLY, DONE, XXX} state_e;  
endpackage
```

```
package fsm1_pkg;  
    typedef enum logic [1:0] {IDLE = 2'b00,  
                               READ  = 2'b01,  
                               DLY   = 2'b11,  
                               DONE  = 2'b10,  
                               XXX   = 'x} state_e;  
endpackage
```



КОНЕЧНЫЕ АВТОМАТЫ. РЕГИСТР СОСТОЯНИЯ

```
always_ff @(posedge clk, negedge rst_n)
    if (!rst_n) state <= IDLE;
    else state <= next;

always_ff @(posedge clk, negedge rst_n) begin
    testbad <= go;
    if (!rst_n) state <= IDLE;
    else state <= next;
end
```

1: Error:

<filename><line number>:

The statements in this 'always' block are outside the scope of the synthesis policy. Only an 'if' statement is allowed at the top level in this always block. (ELAB-302)

2: Error: Cant' read 'sverilog' file <filename> (UID-59)



КОНЕЧНЫЕ АВТОМАТЫ. ЧИТАЕМОСТЬ

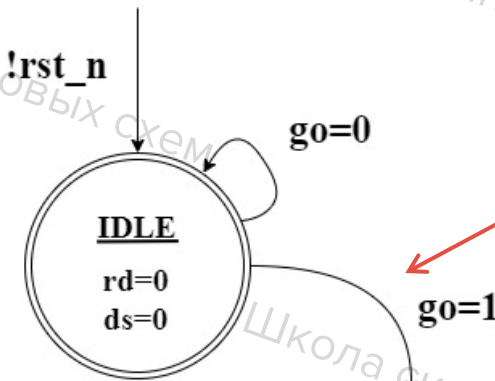
```
always_comb begin
    next = XXX;
    case (state)
        IDLE : if (go) next = READ;
               else next = IDLE; //@ loopback
        READ : next = DLY;
        DLY  : if (!ws) next = DONE;
               else next = READ;
        DONE : next = IDLE;
        default: next = XXX;
    endcase
end
```

```
always_comb begin
    next = XXX;
    case (state)
        IDLE : if (go) next = READ;
               else next = IDLE; //@ loopback
        READ : next = DLY;
        DLY  : if (!ws) next = DONE;
               else next = READ;
        DONE : next = IDLE;
        default: next = XXX;
    endcase
end
```



КОНЕЧНЫЕ АВТОМАТЫ. ПЕТЛЯ В ГРАФЕ СОСТОЯНИЙ

```
always_comb begin
  next = XXX;
  case (state)
    IDLE : if (go) next = READ;
           else next = IDLE; //@ loopback
    READ : next = DLY;
    DLY : if (!ws) next = DONE;
          else next = READ;
    DONE : next = IDLE;
    default: next = XXX;
  endcase
end
```



next = 'x' requires loopback state assignment

loopback state to IDLE

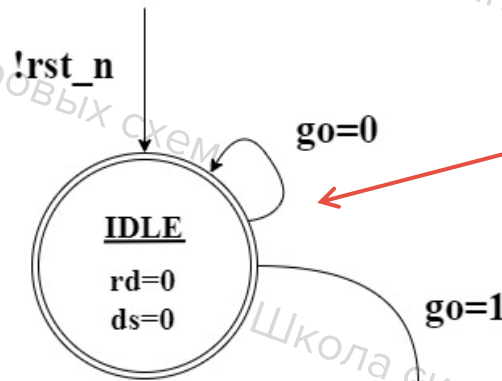


КОНЕЧНЫЕ АВТОМАТЫ. ПЕТЛЯ В ГРАФЕ СОСТОЯНИЙ

```
always_comb begin
    next = XXX;
    case (state)
        IDLE : if (go) next = READ;
              else next = IDLE; //@ loopback
        READ : next = DLY;
        DLY : if (!ws) next = DONE;
              else next = READ;
        DONE : next = IDLE;
        default: next = XXX;
    endcase
end
```

```
always_comb begin
    next = state;
    case (state)
        IDLE : if (go) next = READ;

        READ : next = DLY;
        DLY : if (!ws) next = DONE;
              else next = READ;
        DONE : next = IDLE;
        default: next = XXX;
    endcase
end
```



loopback state to IDLE

next = state requires NO loopback state assignment

УПРАЖНЕНИЕ 2_4_ GAME

 game_config.svh	Renaming in graphics
 game_master_fsm_1_regular_state_encoded.sv	Renaming in graphics
 game_master_fsm_2_special_style_one_hot.sv	Renaming in graphics
 game_master_fsm_3_special_style_signals_from_state.sv	Renaming in graphics
 game_mixer.sv	Renaming in graphics
 game_overlap.sv	Renaming in graphics
 game_random.sv	Renaming in graphics
 game_sprite_control.sv	Renaming in graphics
 game_sprite_display.sv	Renaming in graphics
 game_sprite_display_pipelined.sv	Renaming in graphics
 game_sprite_top.sv	Renaming in graphics
 game_strobe.sv	Renaming in graphics
 game_timer.sv	Renaming in graphics
 game_top.sv	Renaming in graphics
 lab_top.sv	Renaming in graphics
 tb.sv	Renaming in graphics



УПРАЖНЕНИЕ 2_4_GAME

```
game_top
# (
    .clk_mhz          (clk_mhz          ),
    .pixel_mhz        (pixel_mhz        ),
    .screen_width     (screen_width     ),
    .screen_height    (screen_height    ),
    .strobe_to_update_xy_counter_width (strobe_to_update_xy_counter_width)
)
i_game_top
(
    .clk              ( clk              ),
    .rst              ( rst              ),
    .launch_key       ( | key            ),
    .left_right_keys  ( { key [1], key [0] } ),
    .display_on       ( display_on       ),
    .x                 ( x                ),
    .y                 ( y                ),
    .rgb               ( rgb              )
);
```

Вся логика скрыта в модуле game_top. В него параметрами передается тактовая частота в МГц и разрядность счетчика для обновления координат объектов.

Игра запускается по нажатию на любую клавишу. key[1] key[0] управляют движением влево и вправо.

```
assign red    = { 4 { rgb [2] } };
assign green  = { 4 { rgb [1] } };
assign blue   = { 4 { rgb [0] } };
```



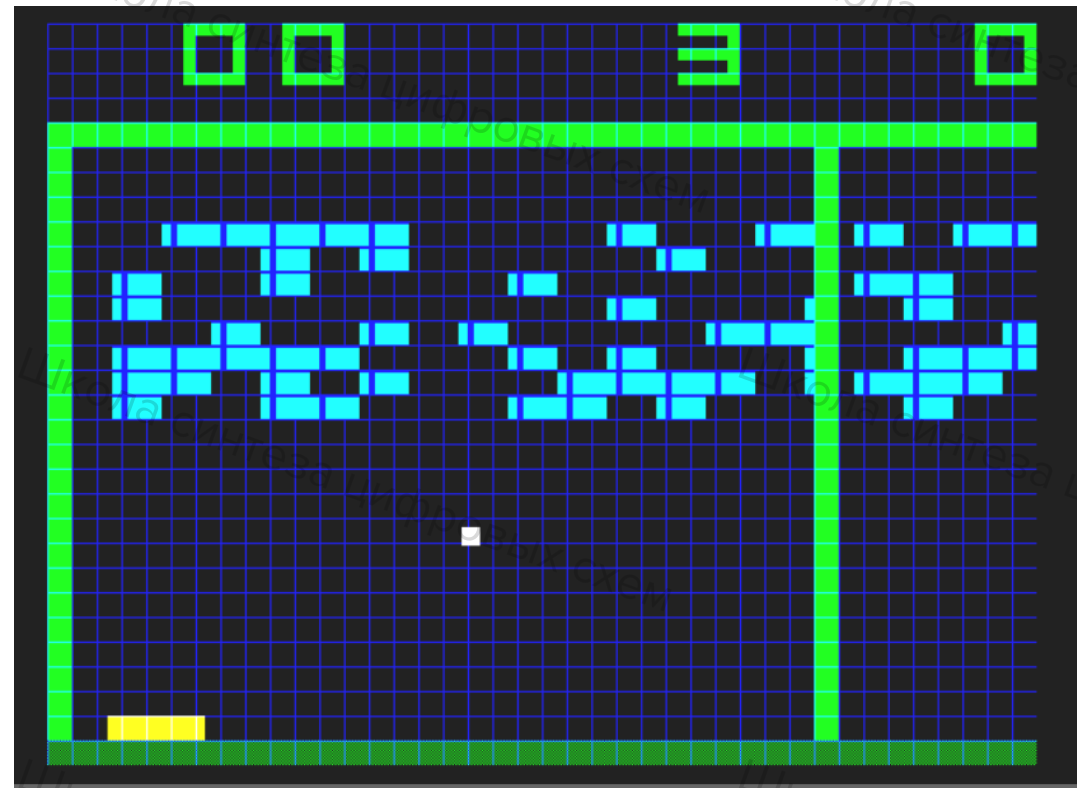
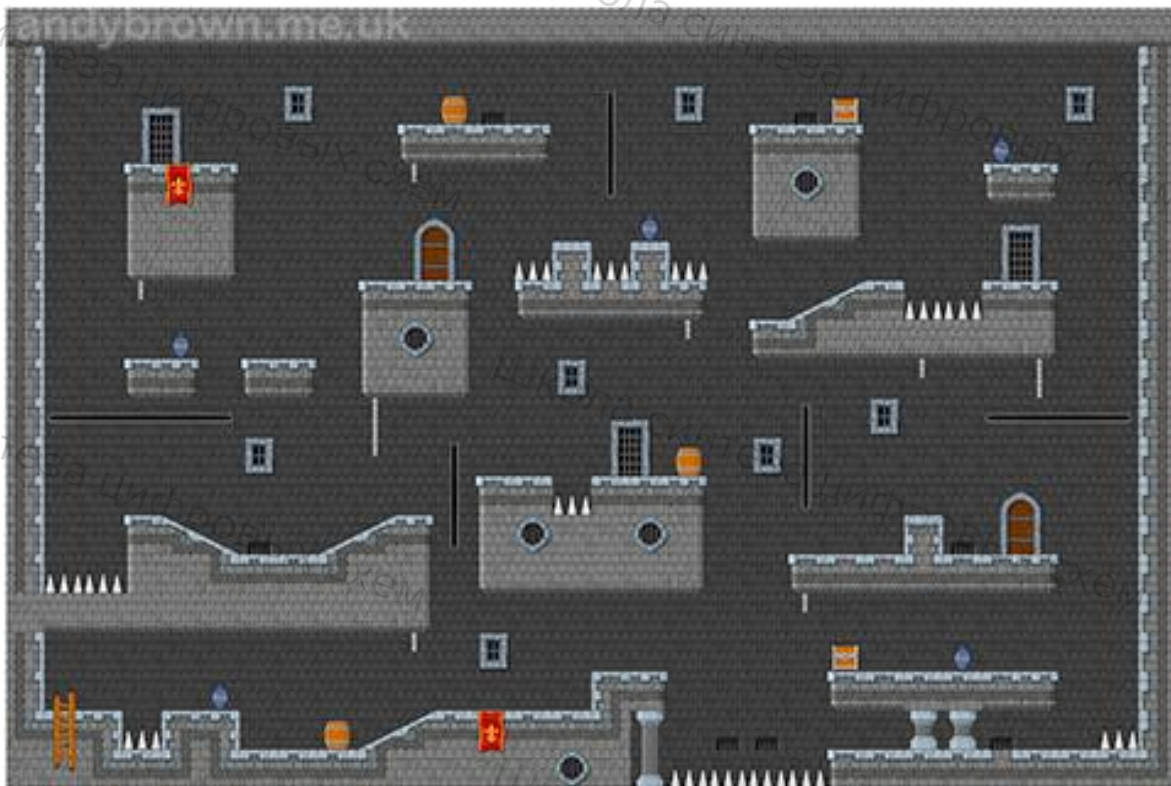
УПРАЖНЕНИЕ 2_4_ GAME

```
vga
# (
  .N_MIXER_PIPE_STAGES ( `N_MIXER_PIPE_STAGES ),
  .HPOS_WIDTH           ( `X_WIDTH           ),
  .VPOS_WIDTH           ( `Y_WIDTH           ),
  .CLK_MHZ               ( clk_mhz           )
)
i_vga
(
  .clk      ( clk      ),
  .rst      ( rst      ),
  .hsync    ( hsync    ),
  .vsync    ( vsync    ),
  .display_on ( display_on ),
  .hpos     ( pixel_x  ),
  .vpos     ( pixel_y  )
);
```

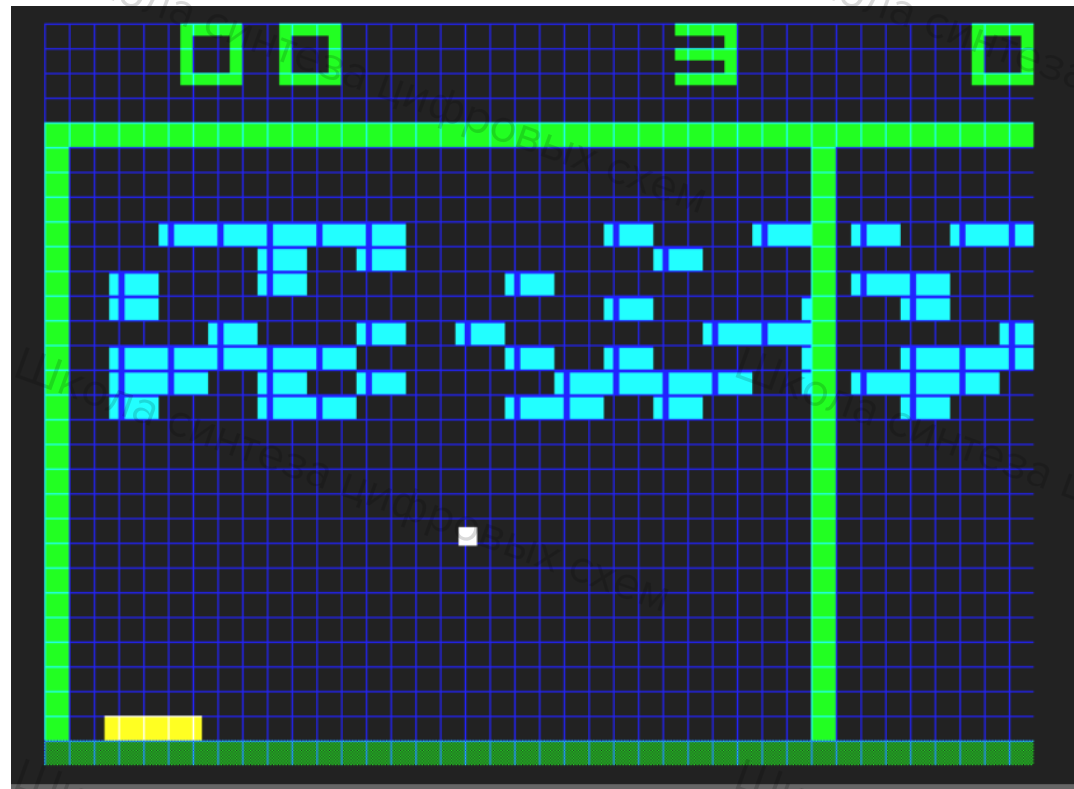
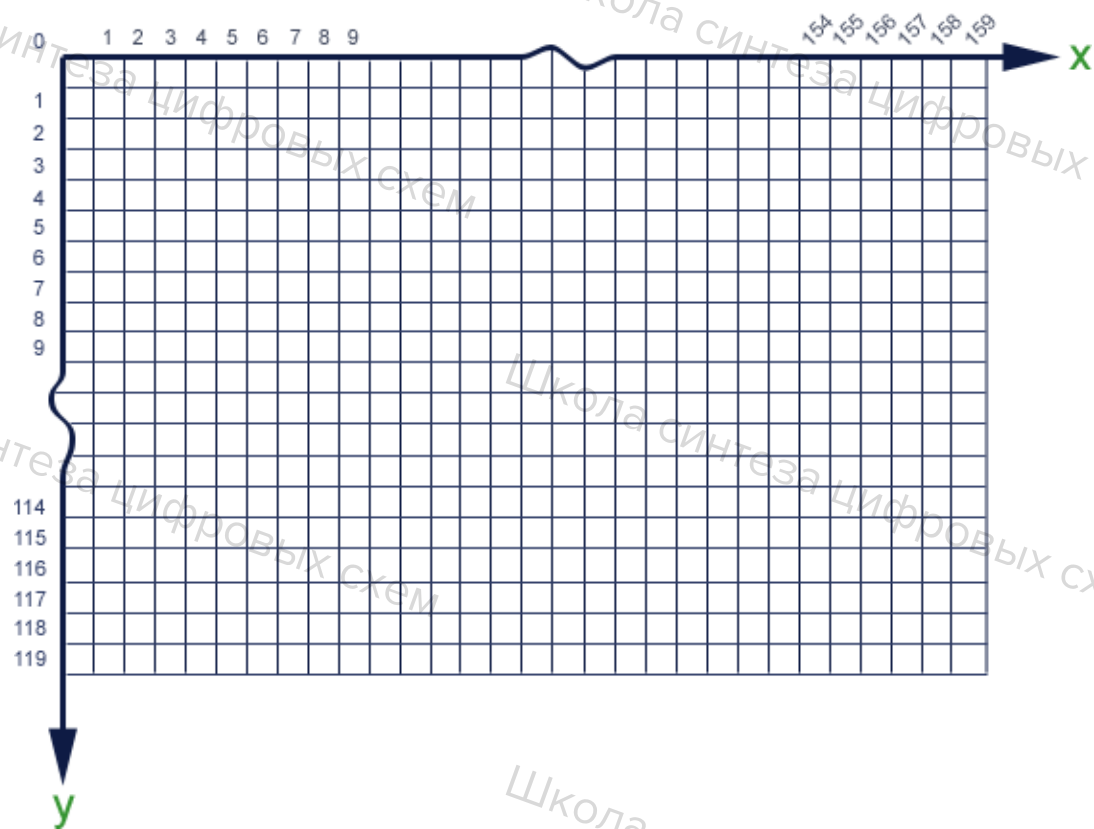
Генератор развертки VGA изображения.
Генерирует сигналы hsync vsync.

Генератор выдает текущие координаты пикселя,
который передается на VGA интерфейс.

УПРАЖНЕНИЕ 2_4_GAME. СПРАЙТОВАЯ ГРАФИКА



УПРАЖНЕНИЕ 2_4_GAME. СПРАЙТОВАЯ ГРАФИКА





УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```
game_sprite_top
#(
    .SPRITE_WIDTH  ( 8 ),
    .SPRITE_HEIGHT ( 8 ),

    .DX_WIDTH      ( 2 ),
    .DY_WIDTH      ( 1 ),

    .ROW_0 ( 32'h000bb000 ),
    .ROW_1 ( 32'h00099000 ),
    .ROW_2 ( 32'h00099000 ),
    .ROW_3 ( 32'hb99ff99b ),
    .ROW_4 ( 32'hb99ff99b ),
    .ROW_5 ( 32'h00099000 ),
    .ROW_6 ( 32'h00099000 ),
    .ROW_7 ( 32'h000bb000 ),

    .screen_width
    (screen_width),

    .screen_height
    (screen_height),

    .strobe_to_update_xy_counter_width
    (strobe_to_update_xy_counter_width)
)
```

Размер спрайта в пикселях.
В данном случае 8 на 8

Разрядность приращения координат
по осям X и Y

Паттерн спрайта. Где каждый пиксель
закодирован 4 битами. ENABLE RED GREEN BLUE
Цвет задается 3 битами. ENABLE отвечает за то,
должен ли пиксель гореть.
Важно при наложении спрайтов друг на друга.



УПРАЖНЕНИЕ 2_4_GAME. GAME_TOP

```
assign sprite_torpedo_write_x = `SCREEN_WIDTH / 2 + random
[15:10];
assign sprite_torpedo_write_y = SCREEN_HEIGHT - 16;
always_comb
begin
    case (left_right_keys)
        2'b00: sprite_torpedo_write_dx = 2'b00;
        2'b01: sprite_torpedo_write_dx = 2'b01;
        2'b10: sprite_torpedo_write_dx = 2'b11;
        2'b11: sprite_torpedo_write_dx = 2'b00;
    endcase
    case (left_right_keys)
        2'b00: sprite_torpedo_write_dy = 3'b111;
        2'b01: sprite_torpedo_write_dy = 3'b110;
        2'b10: sprite_torpedo_write_dy = 3'b110;
        2'b11: sprite_torpedo_write_dy = 3'b110;
    endcase
end
```

Рандомизация начальных координат торпеды по оси **X** для каждой итерации. И фиксированного по оси **Y**

Измерение направления движения по оси в зависимости от нажатых клавиш. Приращения координат **знаковые**. Старший бит знак. 1 — отрицательное приращение. 0 — положительное. По оси **Y** торпеда всегда движется в вверх.



УПРАЖНЕНИЕ 2_4_GAME. GAME_TOP

```
always_comb begin
    if (random [7]) begin
        sprite_target_write_x = 10'd0;
        sprite_target_write_dx = 2'b01;
    end
    else begin
        sprite_target_write_x = `SCREEN_WIDTH - 8;
        sprite_target_write_dx = { 1'b1, random [6] };
    end
end
end
assign sprite_target_write_y = `SCREEN_HEIGHT / 10 + random [5:0];
assign sprite_target_write_dy = 1'd0;
```

Начальное положение объекта выбирается
псевдо-случайно

Объект будет двигаться горизонтально.



УПРАЖНЕНИЕ 2_4_ GAME

Упражнение 1.

Измените изображения объектов подкорректировав параметры страйтов и посмотрите что получится

Упражнение 2.

1. Добавьте рандомизацию для начальных координат торпеды по оси **y**
2. Попробуйте проинвертировать управление.
А проверив, верните его обратно



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

sprite_target
(
    .clk          ( clk          ),
    .rst          ( rst          ),
    .pixel_x      ( pixel_x      ),
    .pixel_y      ( pixel_y      ),
    .sprite_write_xy ( sprite_target_write_xy ),
    .sprite_write_dxy ( sprite_target_write_dxy ),
    .sprite_write_x ( sprite_target_write_x ),
    .sprite_write_y ( sprite_target_write_y ),
    .sprite_write_dx ( sprite_target_write_dx ),
    .sprite_write_dy ( sprite_target_write_dy ),
    .sprite_enable_update ( sprite_target_enable_update ),
    .sprite_x      ( sprite_target_x ),
    .sprite_y      ( sprite_target_y ),
    .sprite_within_screen ( sprite_target_within_screen ),
    .sprite_out_left ( sprite_target_out_left ),
    .sprite_out_right ( sprite_target_out_right ),
    .sprite_out_top ( sprite_target_out_top ),
    .sprite_out_bottom ( sprite_target_out_bottom ),
    .rgb_en        ( sprite_target_rgb_en ),
    .rgb           ( sprite_target_rgb ),
);
    
```

Координаты текущего передаваемого по VGA пикселя



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```
sprite_target
(
    .clk          ( clk          ),
    .rst          ( rst          ),
    .pixel_x      ( pixel_x      ),
    .pixel_y      ( pixel_y      ),
    .sprite_write_xy ( sprite_target_write_xy ),
    .sprite_write_dxy ( sprite_target_write_dxy ),
    .sprite_write_x  ( sprite_target_write_x  ),
    .sprite_write_y  ( sprite_target_write_y  ),
    .sprite_write_dx  ( sprite_target_write_dx ),
    .sprite_write_dy  ( sprite_target_write_dy ),
    .sprite_enable_update ( sprite_target_enable_update ),
    .sprite_x        ( sprite_target_x        ),
    .sprite_y        ( sprite_target_y        ),
    .sprite_within_screen ( sprite_target_within_screen ),
    .sprite_out_left  ( sprite_target_out_left ),
    .sprite_out_right ( sprite_target_out_right ),
    .sprite_out_top   ( sprite_target_out_top  ),
    .sprite_out_bottom ( sprite_target_out_bottom ),
    .rgb_en           ( sprite_target_rgb_en   ),
    .rgb              ( sprite_target_rgb     )
);
```

Координаты текущего передаваемого по VGA пикселя

Сигналы обновления координат и приращений. Приходят от модуля **master_fsm**. Например, когда нужно начать игру заново. Или изменить движение торпеды.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

sprite_target
(
    .clk          ( clk          ),
    .rst          ( rst          ),
    .pixel_x      ( pixel_x      ),
    .pixel_y      ( pixel_y      ),
    .sprite_write_xy ( sprite_target_write_xy ),
    .sprite_write_dxy ( sprite_target_write_dxy ),
    .sprite_write_x ( sprite_target_write_x ),
    .sprite_write_y ( sprite_target_write_y ),
    .sprite_write_dx ( sprite_target_write_dx ),
    .sprite_write_dy ( sprite_target_write_dy ),
    .sprite_enable_update ( sprite_target_enable_update ),
    .sprite_x      ( sprite_target_x ),
    .sprite_y      ( sprite_target_y ),
    .sprite_within_screen ( sprite_target_within_screen ),
    .sprite_out_left ( sprite_target_out_left ),
    .sprite_out_right ( sprite_target_out_right ),
    .sprite_out_top ( sprite_target_out_top ),
    .sprite_out_bottom ( sprite_target_out_bottom ),
    .rgb_en        ( sprite_target_rgb_en ),
    .rgb           ( sprite_target_rgb ),
);
    
```

Координаты текущего передаваемого по VGA пикселя

Сигналы обновления координат и приращений. Приходят от модуля **master_fsm**. Например, когда нужно начать игру заново. Или изменить движение торпеды.

Входные шины с начальными координатами и приращениями



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

sprite_target
(
    .clk          ( clk          ),
    .rst          ( rst          ),
    .pixel_x      ( pixel_x      ),
    .pixel_y      ( pixel_y      ),
    .sprite_write_xy ( sprite_target_write_xy ),
    .sprite_write_dxy ( sprite_target_write_dxy ),
    .sprite_write_x ( sprite_target_write_x ),
    .sprite_write_y ( sprite_target_write_y ),
    .sprite_write_dx ( sprite_target_write_dx ),
    .sprite_write_dy ( sprite_target_write_dy ),
    .sprite_enable_update ( sprite_target_enable_update ),
    .sprite_x      ( sprite_target_x ),
    .sprite_y      ( sprite_target_y ),
    .sprite_within_screen ( sprite_target_within_screen ),
    .sprite_out_left ( sprite_target_out_left ),
    .sprite_out_right ( sprite_target_out_right ),
    .sprite_out_top ( sprite_target_out_top ),
    .sprite_out_bottom ( sprite_target_out_bottom ),
    .rgb_en        ( sprite_target_rgb_en ),
    .rgb           ( sprite_target_rgb ),
);
    
```

Координаты текущего передаваемого по VGA пикселя

Сигналы обновления координат и приращений.
Приходят от модуля **master_fsm**. Например, когда
нужно начать игру заново. Или изменить движение
торпеды.

Входные шины с начальными координатами
и приращениями

Сигнал обновления координат. $X + dX$ $Y + dY$
Приходит от модуля **master_fsm**.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

sprite_target
(
    .clk          ( clk          ),
    .rst          ( rst          ),
    .pixel_x      ( pixel_x      ),
    .pixel_y      ( pixel_y      ),
    .sprite_write_xy ( sprite_target_write_xy ),
    .sprite_write_dxy ( sprite_target_write_dxy ),
    .sprite_write_x ( sprite_target_write_x ),
    .sprite_write_y ( sprite_target_write_y ),
    .sprite_write_dx ( sprite_target_write_dx ),
    .sprite_write_dy ( sprite_target_write_dy ),
    .sprite_enable_update ( sprite_target_enable_update ),
    .sprite_x      ( sprite_target_x ),
    .sprite_y      ( sprite_target_y ),
    .sprite_within_screen ( sprite_target_within_screen ),
    .sprite_out_left ( sprite_target_out_left ),
    .sprite_out_right ( sprite_target_out_right ),
    .sprite_out_top ( sprite_target_out_top ),
    .sprite_out_bottom ( sprite_target_out_bottom ),
    .rgb_en        ( sprite_target_rgb_en ),
    .rgb           ( sprite_target_rgb ),
);
    
```

Координаты текущего передаваемого по VGA пикселя

Сигналы обновления координат и приращений. Приходят от модуля **master_fsm**. Например, когда нужно начать игру заново. Или изменить движение торпеды.

Входные шины с начальными координатами и приращениями

Сигнал обновления координат. $X + dX$ $Y + dY$. Приходит от модуля **master_fsm**.

Шины передающие в модуль **master_fsm** текущие координаты спрайта.

УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

sprite_target
(
  .clk          ( clk          ),
  .rst          ( rst          ),
  .pixel_x      ( pixel_x      ),
  .pixel_y      ( pixel_y      ),
  .sprite_write_xy ( sprite_target_write_xy ),
  .sprite_write_dxy ( sprite_target_write_dxy ),
  .sprite_write_x  ( sprite_target_write_x  ),
  .sprite_write_y  ( sprite_target_write_y  ),
  .sprite_write_dx  ( sprite_target_write_dx ),
  .sprite_write_dy  ( sprite_target_write_dy ),
  .sprite_enable_update ( sprite_target_enable_update ),
  .sprite_x        ( sprite_target_x        ),
  .sprite_y        ( sprite_target_y        ),
  .sprite_within_screen ( sprite_target_within_screen ),
  .sprite_out_left  ( sprite_target_out_left ),
  .sprite_out_right ( sprite_target_out_right ),
  .sprite_out_top   ( sprite_target_out_top  ),
  .sprite_out_bottom ( sprite_target_out_bottom ),
  .rgb_en           ( sprite_target_rgb_en   ),
  .rgb              ( sprite_target_rgb     )
);
    
```

Координаты текущего передаваемого по VGA пикселя

Сигналы обновления координат и приращений. Приходят от модуля **master_fsm**. Например, когда нужно начать игру заново. Или изменить движение торпеды.

Входные шины с начальными координатами и приращениями

Сигнал обновления координат. $X + dX$ $Y + dY$. Приходит от модуля **master_fsm**.

Шины передающие в модуль **master_fsm** текущие координаты спрайта.

Набор сигналов. Определяющие находится ли спрайт в видимой области экрана и его границы



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

sprite_target
(
  .clk          ( clk          ),
  .rst          ( rst          ),
  .pixel_x      ( pixel_x      ),
  .pixel_y      ( pixel_y      ),
  .sprite_write_xy ( sprite_target_write_xy ),
  .sprite_write_dxy ( sprite_target_write_dxy ),
  .sprite_write_x  ( sprite_target_write_x  ),
  .sprite_write_y  ( sprite_target_write_y  ),
  .sprite_write_dx ( sprite_target_write_dx ),
  .sprite_write_dy ( sprite_target_write_dy ),
  .sprite_enable_update ( sprite_target_enable_update ),
  .sprite_x       ( sprite_target_x       ),
  .sprite_y       ( sprite_target_y       ),
  .sprite_within_screen ( sprite_target_within_screen ),
  .sprite_out_left  ( sprite_target_out_left  ),
  .sprite_out_right ( sprite_target_out_right ),
  .sprite_out_top   ( sprite_target_out_top   ),
  .sprite_out_bottom ( sprite_target_out_bottom ),
  .rgb_en          ( sprite_target_rgb_en ),
  .rgb             ( sprite_target_rgb ),
);
    
```

Координаты текущего передаваемого по VGA пикселя

Сигналы обновления координат и приращений. Приходят от модуля **master_fsm**. Например, когда нужно начать игру заново. Или изменить движение торпеды.

Входные шины с начальными координатами и приращениями

Сигнал обновления координат. $X + dX$ $Y + dY$. Приходит от модуля **master_fsm**.

Шины передающие в модуль **master_fsm** текущие координаты спрайта.

Набор сигналов. Определяющие находится ли спрайт в видимой области экрана и его границы

Выходная информация о пикселе. Нужно ли его рисовать и цвет. Определяется через сравнение координат спрайта и координат текущего пикселя для VGA



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```
game_overlap overlap
(
    .clk      ( clk
    .rst      ( rst
    .left_1   ( sprite_target_out_left
    .right_1  ( sprite_target_out_right
    .top_1    ( sprite_target_out_top
    .bottom_1 ( sprite_target_out_bottom
    .left_2   ( sprite_torpedo_out_left
    .right_2  ( sprite_torpedo_out_right
    .top_2    ( sprite_torpedo_out_top
    .bottom_2 ( sprite_torpedo_out_bottom
    .overlap  ( collision
)
```

Определяет момент пересечений двух спрайтов на экране по сравнению их текущих координат



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

wire end_of_game_timer_start;
wire end_of_game_timer_running;
game_timer # (.width (25)) timer
(
    .clk      ( clk                ),
    .rst      ( rst                ),
    .value    ( 25'h1000000        ),
    .start    ( end_of_game_timer_start ),
    .running  ( end_of_game_timer_running )
);
    
```

Таймер, отсчитывающий время в игре.
Время задано параметром в тактах.
Запускается конечным автоматом в `master_fsm`.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```
game_mixer mixer
(
    .clk            ( clk            ),
    .rst            ( rst            ),

    .display_on     ( display_on     ),

    .sprite_target_rgb_en ( sprite_target_rgb_en ),
    .sprite_target_rgb   ( sprite_target_rgb   ),

    .sprite_torpedo_rgb_en ( sprite_torpedo_rgb_en ),
    .sprite_torpedo_rgb   ( sprite_torpedo_rgb   ),

    .game_won        ( game_won        ),
    .end_of_game_timer_running ( end_of_game_timer_running ),

    .random           ( random [0]      ),
    .rgb               ( rgb            )
);
```

Модуль, отвечающий за вывод пикселей на экран от разных спрайтов и состояний игры.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_MIXER

```
always_ff @ (posedge clk or posedge rst)
  if (rst)
    rgb <= 3'b000;
  else if (!display_on)
    rgb <= 3'b000;
  else if (end_of_game_timer_running)
    rgb <= { 1'b1, ~ game_won, random };
  else if (sprite_torpedo_rgb_en)
    rgb <= sprite_torpedo_rgb;
  else if (sprite_target_rgb_en)
    rgb <= sprite_target_rgb;
  else
    rgb <= 3'b000;
```

Самые приоритетные изображения:

- Черный экран;
- Завершение игры;
- Спрайт торпеды;
- Спрайт объекта.



УПРАЖНЕНИЕ 2_4_ GAME

Упражнение 3.

Давайте изменим вывод графической информации.

Измените картинку выводящуюся при завершении игры



УПРАЖНЕНИЕ 2_4_ GAME. GAME_TOP

```

`GAME_MASTER_FSM_MODULE master_fsm
(
    .clk                ( clk                ),
    .rst                ( rst                ),
    .launch_key         ( launch_key         ),
    .sprite_target_write_xy ( sprite_target_write_xy ),
    .sprite_torpedo_write_xy ( sprite_torpedo_write_xy ),
    .sprite_target_write_dxy ( sprite_target_write_dxy ),
    .sprite_torpedo_write_dxy ( sprite_torpedo_write_dxy ),
    .sprite_target_enable_update ( sprite_target_enable_update ),
    .sprite_torpedo_enable_update ( sprite_torpedo_enable_update ),
    .sprite_target_within_screen ( sprite_target_within_screen ),
    .sprite_torpedo_within_screen ( sprite_torpedo_within_screen ),
    .collision           ( collision           ),
    .game_won            ( game_won            ),
    .end_of_game_timer_start ( end_of_game_timer_start ),
    .end_of_game_timer_running ( end_of_game_timer_running ),
);
    
```

Главный конечный автомат, отвечающий за обработку логики игры.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_MASTER_FSM

```
localparam [1:0] STATE_START = 0,
               STATE_AIM     = 1,
               STATE_SHOOT    = 2,
               STATE_END      = 3;
```

```
logic [1:0] state;
logic [1:0] d_state;
```

Вся логика игры описывается в 4 состояниях:

- START
- AIM
- SHOOT
- END

Ко всем параметрам, описывающим состояния, добавляется префикс **STATE** для большей читаемости. Это очень удобно при большом количестве сигналов и параметров.

Так же для читаемости **ПАРАМЕТРЫ** и **DEFINE** пишут в верхнем регистре, а сигналы — в нижнем.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_MASTER_FSM

```
STATE_START:  
begin  
    d_sprite_target_write_xy = 1'b1;  
    d_sprite_torpedo_write_xy = 1'b1;  
    d_sprite_target_write_dxy = 1'b1;  
    d_game_won = 1'b0;  
    d_state = STATE_AIM;  
end
```

Начальное состояние конечного автомата.
Инициализация игры.

На следующий такт происходит переход
в состояние AIM.

Обновляются координаты торпеды и объекта,
сбрасывается статус победы, обновляются
приращения, задающие движение объекта.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_MASTER_FSM

```
STATE_AIM:
begin
    d_sprite_target_enable_update = 1'b1;
    if (end_of_game)
    begin
        d_end_of_game_timer_start = 1'b1;
        d_state = STATE_END;
    end
    else if (launch_key)
    begin
        d_state = STATE_SHOOT;
    end
end
```

Состояние ожидания начала игры через нажатия любой кнопки.

По экрану движется только объект.

Игра завершается, если закончилось время.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_MASTER_FSM

```
STATE_SHOOT:
begin
    d_sprite_torpedo_write_dxy      = 1'b1;
    d_sprite_target_enable_update   = 1'b1;
    d_sprite_torpedo_enable_update  = 1'b1;
    if (collision)
        d_game_won = 1'b1;
    if (end_of_game)
        begin
            d_end_of_game_timer_start = 1'b1;
            d_state                    = STATE_END;
        end
    end
end
```

Состояние, в котором движется
и торпеда, и объект.

Игра завершается либо при пересечении
спрайтов, либо по завершению счета таймера.



УПРАЖНЕНИЕ 2_4_ GAME. GAME_MASTER_FSM

```
STATE_END:
begin
// TODO: Investigate why it needs collision detection here
// and not in previous state
if (collision)
    d_game_won = 1'b1;
if (! end_of_game_timer_running)
    d_state = STATE_START;
end
```




УПРАЖНЕНИЕ 2_4_ GAME. GAME_GAME_CONFIG

```
`ifndef GAME_CONFIG_SVH
`define GAME_CONFIG_SVH

`include "config.svh"

`define SCREEN_WIDTH 640
`define SCREEN_HEIGHT 480

`define X_WIDTH 10 // X coordinate width in bits
`define Y_WIDTH 10 // Y coordinate width in bits

`define RGB_WIDTH 3

`ifndef GAME_MASTER_FSM_MODULE
`define GAME_MASTER_FSM_MODULE game_master_fsm_1_regular_state_encoded
// `define GAME_MASTER_FSM_MODULE game_master_fsm_2_special_style_one_hot
// `define GAME_MASTER_FSM_MODULE game_master_fsm_3_special_style_signals_from_state
`endif

`define N_MIXER_PIPE_STAGES 1

// `define GAME_SPRITE_DISPLAY_MODULE game_sprite_display_pipelined
`define GAME_SPRITE_DISPLAY_MODULE game_sprite_display

`endif
```

В упражнении есть три реализации конечного автомата игровой логики разными способами и две реализации спрайта.

Два define указывают, какая из реализаций сейчас используется в дизайне.

Кроме того, задается размер изображения, разрядность цвета и разрядность счетчиков координат.



УПРАЖНЕНИЕ 2_4_GAME

Упражнение 4.

Попробуйте модифицировать логику конечного автомата.

Так же выясните почему необходимо обнаружение столкновений
в стадии **STATE_END**



УПРАЖНЕНИЕ 2_4_ GAME. RANDOM

Модуль генерации псевдослучайной последовательности чисел.

```

wire [15:0] random;
game_random random_generator (clk, rst, random);

module game_random
(
    input  clk,
    input  rst,
    output logic [15:0] random
);
    // Uses LFSR, Linear Feedback Shift Register
    always_ff @ (posedge clk or posedge rst)
        if (rst)
            random <= 16'b1111111111111111;
        else
            random <= { random [14:0], 1'b0 }
                ^ ( random [15] ? 16'b1000000001011 : 16'b0);
endmodule
    
```



ГЕНЕРАЦИЯ ПСЕВДОСЛУЧАЙНОЙ ПОСЛЕДОВАТЕЛЬНОСТИ ЧИСЕЛ В RTL

Материалы для этой части презентации взяты из лекции
Patrick Schaumont ECE 4514 Digital Design II Lecture 6:
A Random Number Generator in Verilog

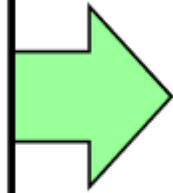
Про аппаратный генератор чисел читал лекцию Илья Кудрявцев
из Самарского университета. Очень интересно, посмотрите.
[youtu.be](https://youtu.be/0333333333)





ГЕНЕРАЦИЯ ПСЕВДОСЛУЧАЙНОЙ ПОСЛЕДОВАТЕЛЬНОСТИ ЧИСЕЛ В RTL

Random
Number
Generator



11, 86, 82, 52, 60, 46, 64, 10, 98, 2, ...

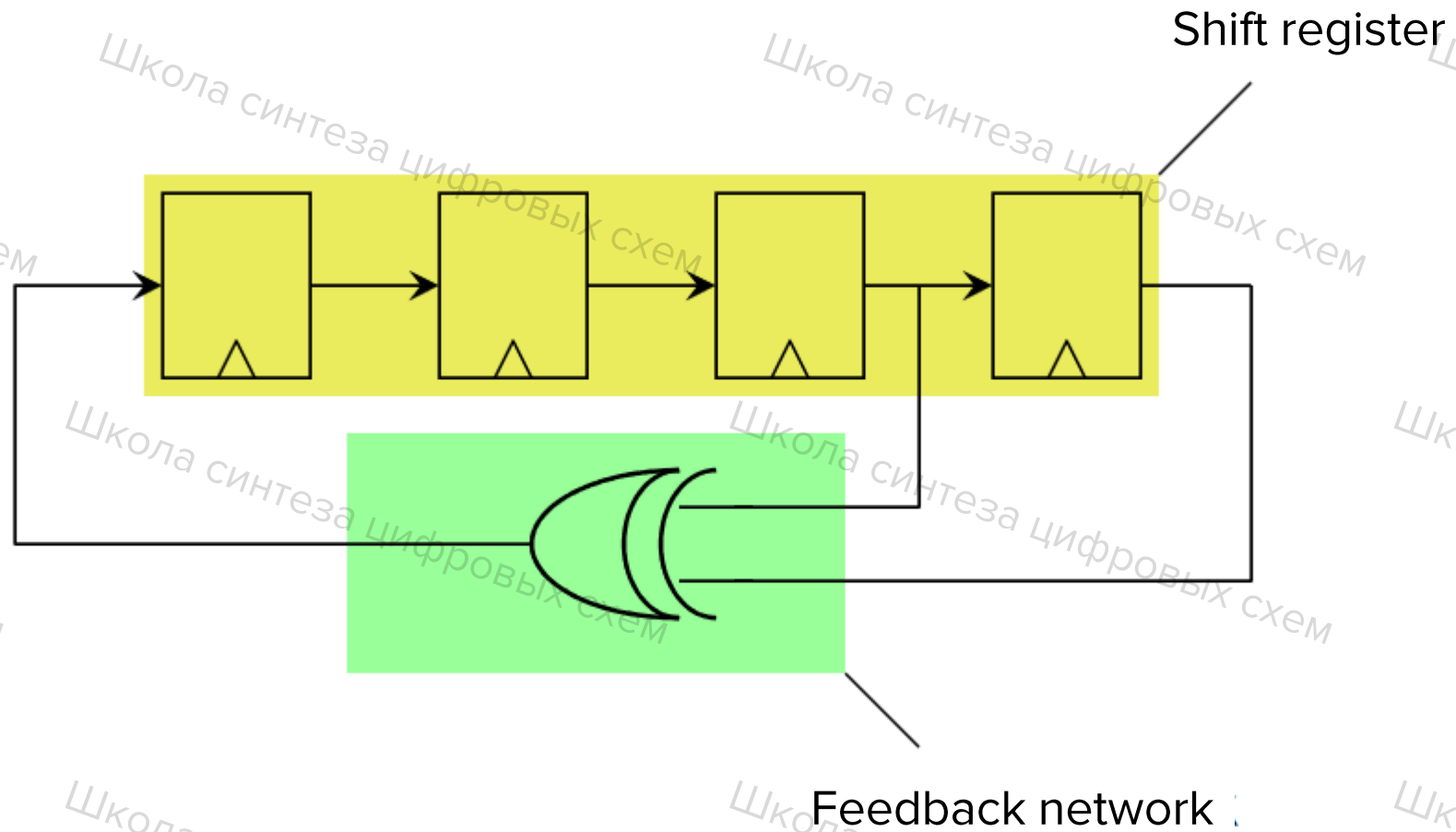




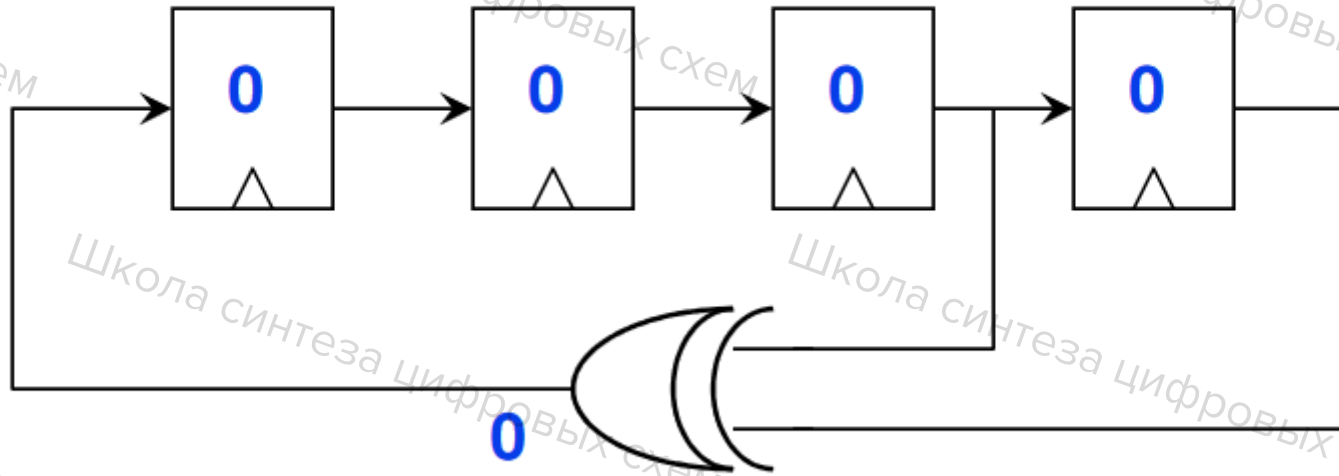
ГЕНЕРАЦИЯ ПСЕВДОСЛУЧАЙНОЙ ПОСЛЕДОВАТЕЛЬНОСТИ ЧИСЕЛ В RTL

```
module random(q);  
    output [0:31] q;  
    reg [0:31] g;  
  
    initial  
        r_seed = 2;  
    always  
        #10 q = $random(r_seed);  
endmodule
```

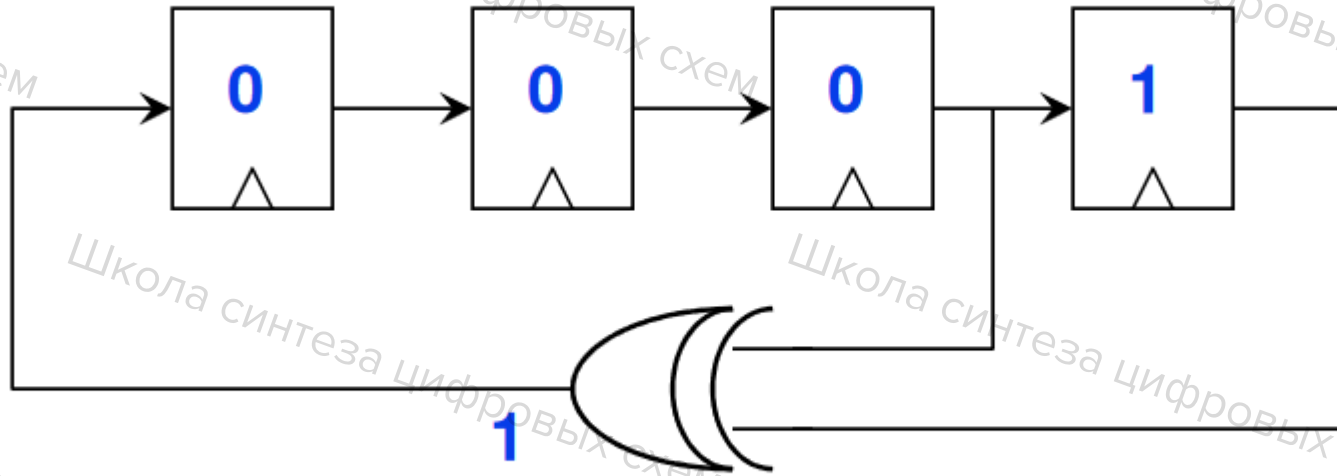
LINEAR FEEDBACK SHIFT REGISTER



LINEAR FEEDBACK SHIFT REGISTER

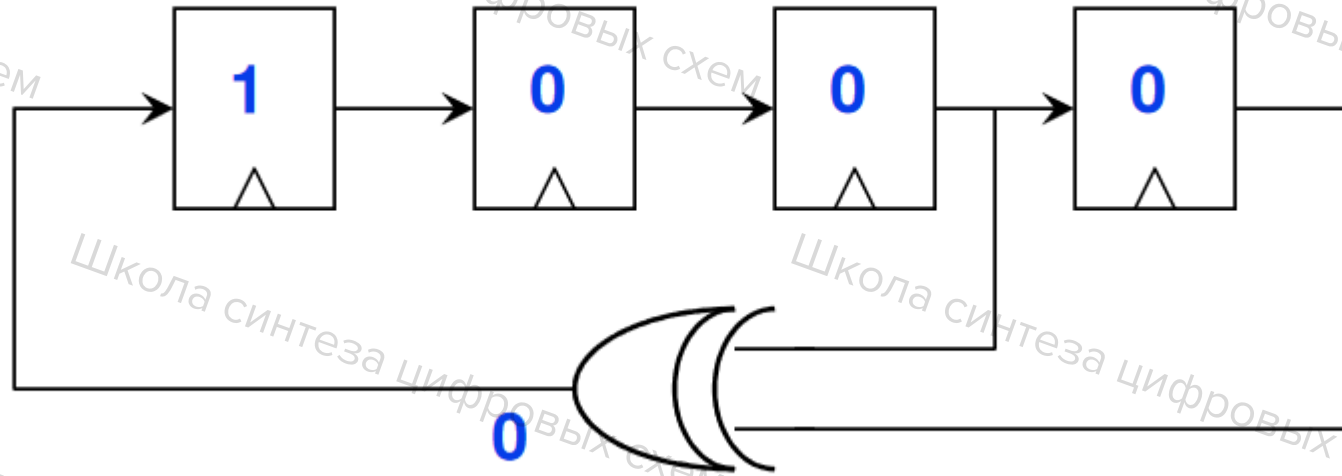


LINEAR FEEDBACK SHIFT REGISTER

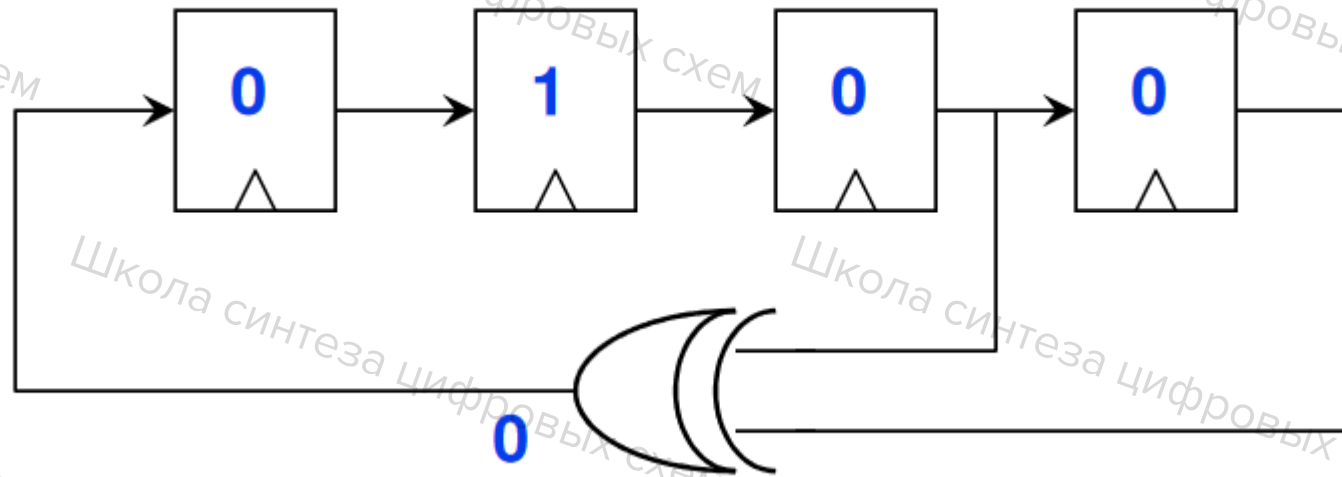




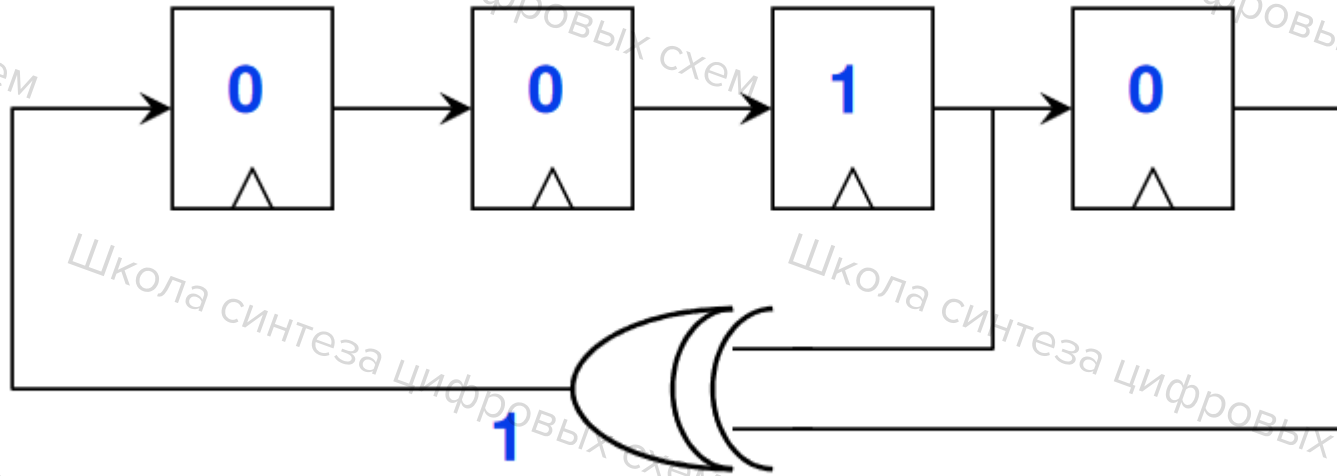
LINEAR FEEDBACK SHIFT REGISTER



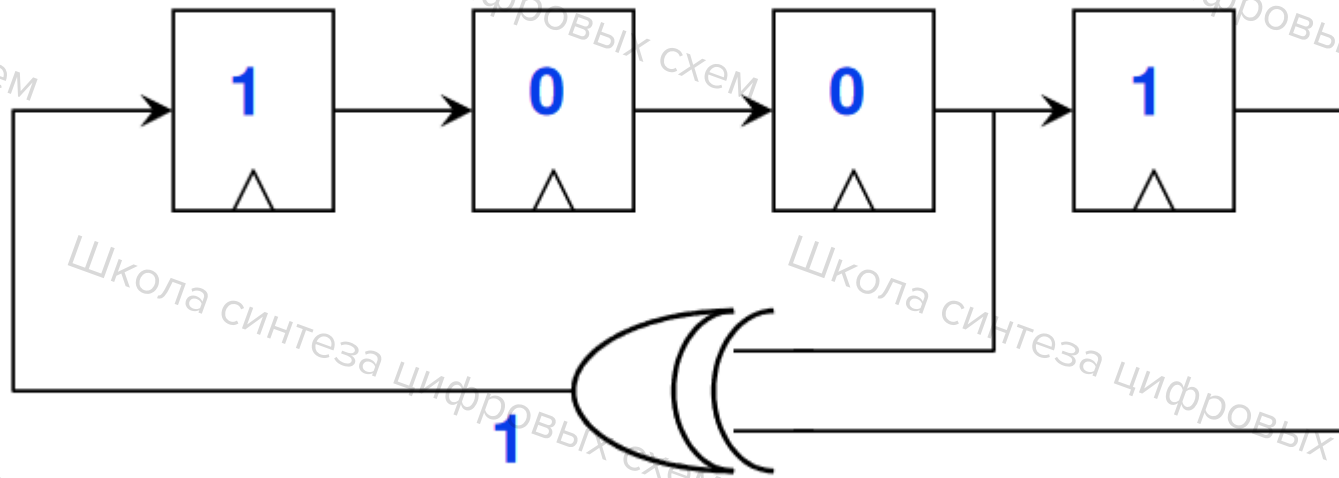
LINEAR FEEDBACK SHIFT REGISTER



LINEAR FEEDBACK SHIFT REGISTER

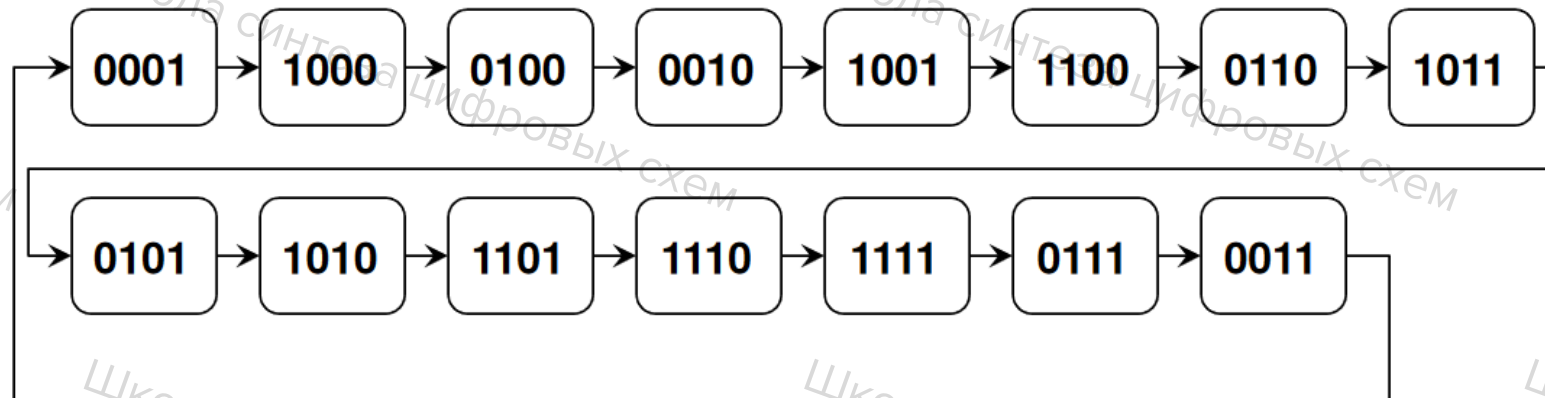
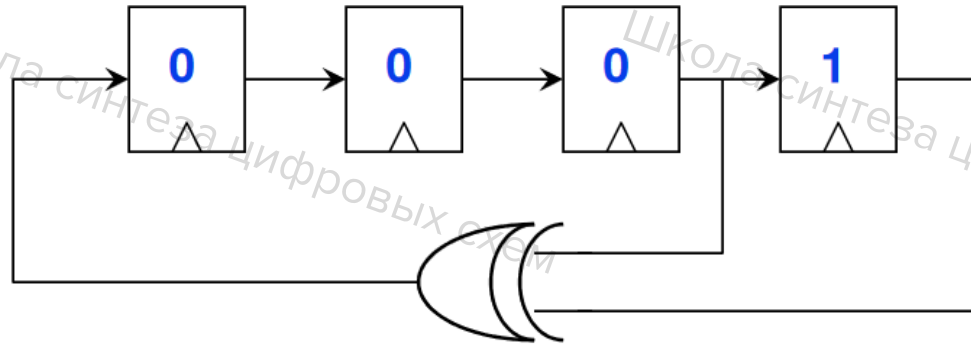


LINEAR FEEDBACK SHIFT REGISTER



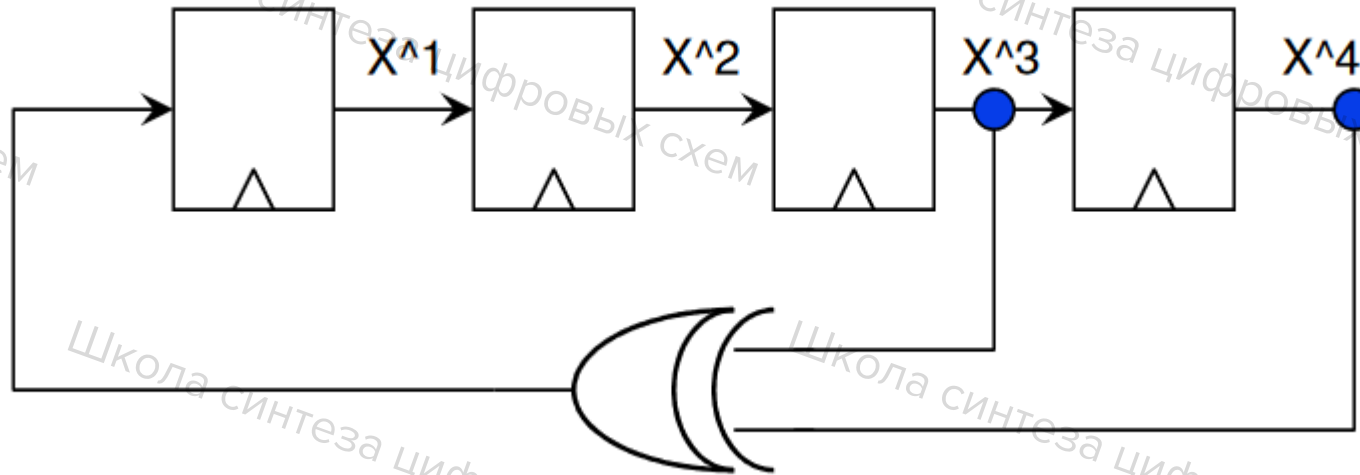


LINEAR FEEDBACK SHIFT REGISTER





LINEAR FEEDBACK SHIFT REGISTER



$$P(x) = x^4 + x^3 + 1$$



УПРАЖНЕНИЕ 2_4_GAME

Упражнение 5.

Попробуйте модифицировать модуль генерации псевдослучайной последовательности. И посмотрите как будет вести себя игра.
Что изменится?

Упражнение 6.

Внесите изменение в игру по заданию куратора.
И покажите ему результат.